

(1) Written Problems: Batch Size vs. Learning Rate

(a) As batch size increases, how do you expect the optimal learning rate to change?
As batch size increases, I expect the optimal learning rate to also increase. This is because larger batch sizes provide a more accurate estimate of the gradient, allowing for larger steps to be taken in the parameter space without overshooting the minimum.

1.1 Training steps vs. batch size

(a) For the three points (A, B, C) , in Figure 1, which one has the most efficient batch size. Point C has the most efficient batch size. Let's assume that point C is the batch size s.t. the compute is used quasi-optimally. This means that as batch size drastically decreases, the number of batches needed to cover the entire dataset surpasses the number of available compute units, leading to underutilization of resources (point A resides in this region of the chart - many more training steps are required). On the other hand, point B's batch sizes are so large that they exceed the capacity of the compute units, leading to inefficiencies due to memory constraints and increased communication overhead.

(b) Fill in the blanks.

Point A: Regime: Noise-dominated Potential acceleration: Increase batch size

Point B: Regime: Curvature-dominated Potential acceleration: Decrease batch size

(1.2) Model size, dataset size, and compute

(a) Best way to improve model **test** performance (C) Increase the model size. With more PF-days of compute, increasing the model size allows for capturing more complex patterns in the data, leading to better generalization and improved test performance.

Assignment-more-epochs

November 21, 2025

```
[1]: %reload_ext autoreload
      %autoreload 2
```

```
[2]: import torch
      import torch.nn as nn
      from torch.nn import functional as F
      import math
      from torch.nn import LayerNorm
```

```
[3]: from kret_studies import *
      from kret_studies.notebook import *
      from kret_studies.complex import *
```

Loaded environment variables from
/Users/Akseldkw/Desktop/Columbia/COMS4776/.env.

INFO:datasets:JAX version 0.7.2 available.

```
[4]: from kret_studies.kret_torch.mixin.constants import DEVICE_TORCH_STR,
      ↪DEVICE_LITERAL
```

```
device: DEVICE_LITERAL = DEVICE_TORCH_STR
NUM_EPOCHS = 50
```

```
[5]: from trainer import save_model_auto, load_model_auto, model_filename
```

```
[ ]: # !pip install thop
      # !pip install tqdm
```

1 Part 1: Attention and Encoder-Decoder Models

In this section, you'll implement two forms of attention and use them in an encoder-decoder style transformer. You'll then compare performance based on changing dataset size and hidden dimension.

1.1 Step 1: Scaled Dot Product Attention

Implement both the base `scaled_dot_attention` method as well as the `forward` method for the Attention module.

```
[7]: def scaled_dot_attention(
    q: torch.Tensor, k: torch.Tensor, v: torch.Tensor, mask: torch.Tensor | None
    ↪ None = None
):
    """Computes scaled dot product attention with an optional mask.
    q : shape ( batch, k, hidden_size)
    k : shape ( batch, seq_len, hidden_size)
    v : shape ( batch, seq_len, hidden_size)
    mask: optional. shape (k, seq_len)

    returns:
        context : shape (batch, k, hidden_size)
        attention : shape (batch, k, seq_len)
    """

    # -----
    # FILL THIS IN
    # -----
    unnorm_attn = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(q.size(-1))
    if mask is not None:
        unnorm_attn = unnorm_attn.masked_fill(mask == 0, float("-inf"))
    attention = F.softmax(unnorm_attn, dim=-1)
    context = torch.matmul(attention, v)
    return context, attention
```

```
[8]: class Attention(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.Q = nn.Linear(hidden_size, hidden_size, bias=False)
        self.K = nn.Linear(hidden_size, hidden_size, bias=False)
        self.V = nn.Linear(hidden_size, hidden_size, bias=False)
        self.hidden_size = hidden_size
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor, annots: torch.Tensor):
        # x : shape ( batch, k, hidden_size)
        # annots : shape ( batch, seq_len, hidden_size)

        # -----
        # FILL THIS IN
        # Pay attention to which inputs you use for the queries, keys, and values.
        ↪ values.
        # -----
        # q =
        # k =
        # v =
```

```

q = self.Q(x)
k = self.K(annots)
v = self.V(annots)
return scaled_dot_attention(q, k, v)

```

1.2 Step 2: Causal Scaled Dot Product Attention

Using your `scaled_dot_attention` method, implement the `forward` method for the `CausalAttention` module.

```

[9]: class CausalAttention(Attention):
    def __init__(self, hidden_size: int):
        super().__init__(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        # x : shape ( batch, seq_len, hidden_size)

        # -----
        # FILL THIS IN
        # This should be very similar to your other attention implementation,
        # with the exception that you only have one set of outputs and you
        # must construct a causal attention mask.
        # -----
        # q =
        # k =
        # v =
        # mask =
        seq_len = x.size(1)
        q = self.Q(x)
        k = self.K(x)
        v = self.V(x)
        mask = torch.tril(torch.ones((seq_len, seq_len), device=x.device)).
        ↪ bool()
        return scaled_dot_attention(q, k, v, mask)

```

Simple test cases

Passing this DOES NOT mean your implementation is correct. But, if this fails, then you definitely did something wrong

```

[10]: cros_attn = Attention(32)
      caus_attn = CausalAttention(32)
      caus_attn.Q = cros_attn.Q
      caus_attn.K = cros_attn.K
      caus_attn.V = cros_attn.V

```

```

inp = torch.randn(4, 8, 32).to(device)
inp2 = torch.randn(4, 16, 32).to(device)

print("Checking consistent shape")
assert cros_attn(inp, inp2)[0].shape == inp.shape
assert caus_attn(inp)[0].shape == inp.shape
assert cros_attn(inp, inp2)[1].shape == (4, 8, 16)
assert caus_attn(inp)[1].shape == (4, 8, 8)

print("Checking attention mask")
assert torch.all(caus_attn(inp)[0][:, -1] == cros_attn(inp, inp)[0][:, -1])

print("Passed")

```

```

Checking consistent shape
Checking attention mask
Passed

```

Read through the following code. Make sure you understand what is going on. This code sets up the encoder-decoder architecture. It requires no additional code.

```

[11]: class PositionalEncoding(nn.Module):
    pe: torch.Tensor

    def __init__(self, hidden_size: int, max_len: int = 1000):
        super().__init__()

        pos = torch.arange(max_len).float().unsqueeze(1)
        dim = torch.arange(hidden_size // 2).float().unsqueeze(0)

        div_term = torch.exp(-math.log(10000.0) * (2 * dim) / hidden_size)
        angle = pos * div_term

        pe = torch.zeros(max_len, hidden_size)
        pe[:, 0::2] = torch.sin(angle)
        pe[:, 1::2] = torch.cos(angle)

        self.register_buffer("pe", pe)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size, max_len=max_len)

    def forward(self, idx: int):
        return self.pe[idx]

```

```

[12]: class MLP(nn.Module):

    def __init__(self, hidden_size: int):

```

```

        super().__init__()
        self.layer1 = nn.Linear(hidden_size, hidden_size, bias=False)
        self.layer2 = nn.Linear(hidden_size, hidden_size, bias=False)
        self.relu = nn.ReLU()
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x

class EncoderBlock(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.norm1 = LayerNorm(hidden_size)
        self.norm2 = LayerNorm(hidden_size)
        self.attn = Attention(hidden_size)
        self.mlp = MLP(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        context, attention = self.attn(self.norm1(x), self.norm1(x))
        x = x + context
        x = x + self.mlp(self.norm2(x))
        return x, attention

class DecoderBlock(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.norm1 = LayerNorm(hidden_size)
        self.norm2 = LayerNorm(hidden_size)
        self.cros_attn = Attention(hidden_size)
        self.self_attn = CausalAttention(hidden_size)
        self.mlp = MLP(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor, annotation: torch.Tensor):
        context, self_attn = self.self_attn(self.norm1(x))
        x = x + context
        context, cros_attn = self.cros_attn(self.norm2(x), annotation)
        x = x + context

```

```

x = x + self.mlp(self.norm2(x))
return x, self_attn, cross_attn

```

```

[13]: class TransformerEncoderDecoder(nn.Module):
    def __init__(
        self, vocab_size: int, hidden_size: int, num_layers: int, dataset_name:
        ↪str
    ):
        super().__init__()

        self.tok_emb = nn.Embedding(vocab_size, hidden_size)
        self.pos_emb = PositionalEncoding(hidden_size)

        self.encoder_blocks = nn.ModuleList(
            [EncoderBlock(hidden_size) for i in range(num_layers)]
        )
        self.decoder_blocks = nn.ModuleList(
            [DecoderBlock(hidden_size) for i in range(num_layers)]
        )

        self.head = nn.Linear(hidden_size, vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight
        # weight tie, the final layer reuses the weights of the embedding layers
        # this reduces the total number of parameters

        self.apply(self._init_weights) # initialize the weights to a small
        ↪number

        self.to(device)
        self.filename = model_filename(
            hidden_size=hidden_size,
            num_layers=num_layers,
            vocab_size=vocab_size,
            dataset_name=dataset_name,
        )

    def forward(self, inputs: torch.Tensor, annots: torch.Tensor):

        pos_x = torch.arange(
            0, inputs.shape[-1], device=inputs.device
        ).long() # creates tensor([0, 1, 2, 3, ...])
        pos_y = torch.arange(
            0, annots.shape[-1], device=inputs.device
        ).long() # creates tensor([0, 1, 2, 3, ...])

        x = self.tok_emb(inputs) + self.pos_emb(pos_x)
        y = self.tok_emb(annots) + self.pos_emb(pos_y)

```



```

self_attn = []
cros_attn = []

for encode in self.encoder_blocks:
    y, _ = encode(y)

for decode in self.decoder_blocks:
    x, s, c = decode(x, y)
    cros_attn.append(c)
    self_attn.append(s)

return self.head(x), {"self_attn": self_attn, "cros_attn": cros_attn}

def _init_weights(self, module: torch.nn.Module):
    if isinstance(module, nn.Linear):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

```

1.3 Step 3: Training the model

You'll now train a set of example models. This section requires no additional code from you, with the exception of setting the datapath. However, **you are required to analyze and include some of its output.**

```

[14]: from dataset import EncoderDecoderDataset, DecoderDataset
      from utils import save_loss_comparison_by_dataset, □
      ↪ save_loss_comparison_by_hidden
      from torch.utils.data import DataLoader
      from trainer import train
      import torch.optim as optim

data_path = str("./") # Change data_path to where the data is stored

encdec_ds_small = EncoderDecoderDataset(data_path, "pig_latin_small")
encdec_ds_large = EncoderDecoderDataset(data_path, "pig_latin_large")

[15]: print("Dataset sample:")
      print("Input:", encdec_ds_small.to_token(encdec_ds_small[0][0].tolist()))
      print("Annotation:", encdec_ds_small.to_token(encdec_ds_small[0][1].tolist()))
      print("Target:", encdec_ds_small.to_token(encdec_ds_small[0][2].tolist()))

```

```

Dataset sample:
Input: <SOS>attacksway
Annotation: attacks

```

Target: attacksway<EOS>

```
[16]: # model = load_model_auto(  
#     TransformerEncoderDecoder,  
#     vocab_size=len(encdec_ds_small.idx_to_token),  
#     hidden_size=16,  
#     num_layers=4,  
#     dataset_name="small",  
# )  
model = TransformerEncoderDecoder(  
    vocab_size=len(encdec_ds_small.idx_to_token),  
    hidden_size=16,  
    num_layers=4,  
    dataset_name="small",  
)  
  
[17]: optimizer = optim.AdamW(model.parameters(), lr=1e-2)  
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)  
  
results = train(  
    model=model,  
    dataset=encdec_ds_small,  
    num_epochs=NUM_EPOCHS,  
    optimizer=optimizer,  
    scheduler=scheduler,  
    device=device,  
    batch_size=32,  
)  
save_model_auto(model)
```

```
Epoch 1 | Train loss: 2.611418 | Val loss 2.372160  
Epoch 2 | Train loss: 2.130110 | Val loss 1.988386  
Epoch 3 | Train loss: 1.840955 | Val loss 1.654330  
Epoch 4 | Train loss: 1.400597 | Val loss 1.170410  
Epoch 5 | Train loss: 0.924962 | Val loss 0.772506  
Epoch 6 | Train loss: 0.642774 | Val loss 0.565614  
Epoch 7 | Train loss: 0.429669 | Val loss 0.418405  
Epoch 8 | Train loss: 0.347020 | Val loss 0.412847  
Epoch 9 | Train loss: 0.278721 | Val loss 0.262245  
Epoch 10 | Train loss: 0.219025 | Val loss 0.256088  
Epoch 11 | Train loss: 0.195752 | Val loss 0.219199  
Epoch 12 | Train loss: 0.166455 | Val loss 0.232083
```

Epoch 13		Train loss: 0.141823		Val loss 0.164849
Epoch 14		Train loss: 0.140129		Val loss 0.183910
Epoch 15		Train loss: 0.110210		Val loss 0.178727
Epoch 16		Train loss: 0.113148		Val loss 0.142319
Epoch 17		Train loss: 0.095149		Val loss 0.139586
Epoch 18		Train loss: 0.087459		Val loss 0.134540
Epoch 19		Train loss: 0.079461		Val loss 0.112133
Epoch 20		Train loss: 0.069012		Val loss 0.123056
Epoch 21		Train loss: 0.068657		Val loss 0.105797
Epoch 22		Train loss: 0.055821		Val loss 0.110392
Epoch 23		Train loss: 0.049400		Val loss 0.119034
Epoch 24		Train loss: 0.050974		Val loss 0.111201
Epoch 25		Train loss: 0.052771		Val loss 0.107071
Epoch 26		Train loss: 0.052319		Val loss 0.097498
Epoch 27		Train loss: 0.046400		Val loss 0.105196
Epoch 28		Train loss: 0.045447		Val loss 0.102410
Epoch 29		Train loss: 0.037532		Val loss 0.099294
Epoch 30		Train loss: 0.037149		Val loss 0.103136
Epoch 31		Train loss: 0.039326		Val loss 0.094984
Epoch 32		Train loss: 0.032811		Val loss 0.089771
Epoch 33		Train loss: 0.031499		Val loss 0.102703
Epoch 34		Train loss: 0.030762		Val loss 0.094299
Epoch 35		Train loss: 0.027597		Val loss 0.089004
Epoch 36		Train loss: 0.021880		Val loss 0.096011
Epoch 37		Train loss: 0.021241		Val loss 0.103367
Epoch 38		Train loss: 0.021652		Val loss 0.099625
Epoch 39		Train loss: 0.022959		Val loss 0.101854
Epoch 40		Train loss: 0.019153		Val loss 0.091643
Epoch 41		Train loss: 0.016738		Val loss 0.104681
Epoch 42		Train loss: 0.016059		Val loss 0.101920
Epoch 43		Train loss: 0.017403		Val loss 0.110650
Epoch 44		Train loss: 0.014808		Val loss 0.106382

```
Epoch 45 | Train loss: 0.015329 | Val loss 0.108063
Epoch 46 | Train loss: 0.014675 | Val loss 0.101264
Epoch 47 | Train loss: 0.012742 | Val loss 0.106851
Epoch 48 | Train loss: 0.015495 | Val loss 0.120200
Epoch 49 | Train loss: 0.015922 | Val loss 0.115641
Epoch 50 | Train loss: 0.011851 | Val loss 0.118588
```

```
[17]: PosixPath('/Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/TransformerEncoderDecoder/50_epochs/Model_ds_namesmall_vo
cab30_hid16_layers4.pt')
```

```
[18]: def encdec_generate(
    input: str,
    model: TransformerEncoderDecoder,
    dataset: EncoderDecoderDataset,
    max_len: int = 20,
):
    gen_string = ""
    idx = torch.tensor([dataset.start_idx]).to(device)
    annots = torch.tensor(
        [dataset.to_idx(input) + [dataset.end_idx] * (max(0, 12 - len(input)))]
    ).to(device)
    # pad the input string with <EOS> since the collate_fn had to pad the
    ↪ strings

    for i in range(max_len):
        logits = model(idx, annots)[0]

        token_idx: torch.Tensor = logits[0, -1:].argmax(dim=-1)
        idx = torch.cat([idx, token_idx.unsqueeze(0)], dim=1)

        if token_idx.item() == dataset.end_idx:
            break
        else:
            gen_string = gen_string + dataset.idx_to_token[int(token_idx.
            ↪ item())]
    return gen_string

def encdec_translate(
    sentence: str, model: TransformerEncoderDecoder, dataset:
    ↪ EncoderDecoderDataset
):
    words = sentence.split()
    return " ".join([encdec_generate(w, model, dataset) for w in words])
```

```
[19]: encdec_translate("the", model, encdec_ds_small)
```

```
[19]: 'ethay'
```

```
[20]: encdec_translate("the air conditioning is working", model, encdec_ds_small)
```

```
[20]: 'ethay airway onditionintcay isway orkingway'
```

```
[21]: encdec_translate("testing some random words", model, encdec_ds_small)
```

```
[21]: 'estingtay omesay andomray ordsway'
```

```
[ ]: configs = [  
    (16, "small", encdec_ds_small),  
    (16, "large", encdec_ds_large),  
    (32, "small", encdec_ds_small),  
    (32, "large", encdec_ds_large),  
]  
  
all_results = {}  
  
for hidden_size, dataset_name, dataset in configs:  
    print(f"\nTraining model: hidden_size={hidden_size},  
↪dataset={dataset_name}")  
  
    # model = load_model_auto(  
    #     TransformerEncoderDecoder,  
    #     vocab_size=len(dataset.idx_to_token),  
    #     hidden_size=hidden_size,  
    #     num_layers=4,  
    #     dataset_name=dataset_name,  
    # )  
    model = TransformerEncoderDecoder(  
        vocab_size=len(dataset.idx_to_token),  
        hidden_size=hidden_size,  
        num_layers=4,  
        dataset_name=dataset_name,  
    )  
  
    optimizer = optim.AdamW(model.parameters(), lr=1e-2)  
    scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)  
  
    results = train(  
        model=model,  
        dataset=dataset,  
        num_epochs=NUM_EPOCHS,  
        optimizer=optimizer,
```

```

        scheduler=scheduler,
        device=device,
        batch_size=32 if dataset_name == "small" else 256,
    )
    all_results[(hidden_size, dataset_name)] = results
    save_model_auto(model)

```

Training model: hidden_size=16, dataset=small

```

Epoch 1 | Train loss: 2.618335 | Val loss 2.285011
Epoch 2 | Train loss: 2.109412 | Val loss 2.020214
Epoch 3 | Train loss: 1.879337 | Val loss 1.763116
Epoch 4 | Train loss: 1.648527 | Val loss 1.586051
Epoch 5 | Train loss: 1.358186 | Val loss 1.264870
Epoch 6 | Train loss: 0.989228 | Val loss 0.808483
Epoch 7 | Train loss: 0.644319 | Val loss 0.621983
Epoch 8 | Train loss: 0.439071 | Val loss 0.482857
Epoch 9 | Train loss: 0.315054 | Val loss 0.327099
Epoch 10 | Train loss: 0.249638 | Val loss 0.259645
Epoch 11 | Train loss: 0.191472 | Val loss 0.225736
Epoch 12 | Train loss: 0.174900 | Val loss 0.184438
Epoch 13 | Train loss: 0.152502 | Val loss 0.252699
Epoch 14 | Train loss: 0.125014 | Val loss 0.179383
Epoch 15 | Train loss: 0.103001 | Val loss 0.141065
Epoch 16 | Train loss: 0.095708 | Val loss 0.133319
Epoch 17 | Train loss: 0.094098 | Val loss 0.118307
Epoch 18 | Train loss: 0.082210 | Val loss 0.137790
Epoch 19 | Train loss: 0.079758 | Val loss 0.136741
Epoch 20 | Train loss: 0.066751 | Val loss 0.144038
Epoch 21 | Train loss: 0.067387 | Val loss 0.108761
Epoch 22 | Train loss: 0.062502 | Val loss 0.112472
Epoch 23 | Train loss: 0.052264 | Val loss 0.112217
Epoch 24 | Train loss: 0.053979 | Val loss 0.102959
Epoch 25 | Train loss: 0.052490 | Val loss 0.093516
Epoch 26 | Train loss: 0.040401 | Val loss 0.099979

```

Epoch 27 | Train loss: 0.041762 | Val loss 0.110348
Epoch 28 | Train loss: 0.038655 | Val loss 0.092781
Epoch 29 | Train loss: 0.035993 | Val loss 0.093862
Epoch 30 | Train loss: 0.034952 | Val loss 0.109555
Epoch 31 | Train loss: 0.029961 | Val loss 0.099329
Epoch 32 | Train loss: 0.029057 | Val loss 0.103393
Epoch 33 | Train loss: 0.027876 | Val loss 0.110448
Epoch 34 | Train loss: 0.027522 | Val loss 0.098490
Epoch 35 | Train loss: 0.023770 | Val loss 0.111967
Epoch 36 | Train loss: 0.024636 | Val loss 0.106604
Epoch 37 | Train loss: 0.026776 | Val loss 0.098505
Epoch 38 | Train loss: 0.026420 | Val loss 0.090643
Epoch 39 | Train loss: 0.019332 | Val loss 0.096713
Epoch 40 | Train loss: 0.020744 | Val loss 0.101591
Epoch 41 | Train loss: 0.018467 | Val loss 0.103635
Epoch 42 | Train loss: 0.019823 | Val loss 0.100927
Epoch 43 | Train loss: 0.015140 | Val loss 0.097970
Epoch 44 | Train loss: 0.013702 | Val loss 0.100525
Epoch 45 | Train loss: 0.013353 | Val loss 0.102385
Epoch 46 | Train loss: 0.013632 | Val loss 0.105636
Epoch 47 | Train loss: 0.014716 | Val loss 0.107783
Epoch 48 | Train loss: 0.012720 | Val loss 0.109159
Epoch 49 | Train loss: 0.011713 | Val loss 0.108303
Epoch 50 | Train loss: 0.010760 | Val loss 0.116966

Training model: hidden_size=16, dataset=large

Epoch 1 | Train loss: 2.579135 | Val loss 2.277744
Epoch 2 | Train loss: 2.092788 | Val loss 1.905247
Epoch 3 | Train loss: 1.723603 | Val loss 1.527164
Epoch 4 | Train loss: 1.366398 | Val loss 1.025585
Epoch 5 | Train loss: 0.851630 | Val loss 0.672703
Epoch 6 | Train loss: 0.449391 | Val loss 0.331142
Epoch 7 | Train loss: 0.335873 | Val loss 0.291372

Epoch 8 | Train loss: 0.216142 | Val loss 0.149520
Epoch 9 | Train loss: 0.192504 | Val loss 0.159118
Epoch 10 | Train loss: 0.133122 | Val loss 0.104527
Epoch 11 | Train loss: 0.096481 | Val loss 0.086694
Epoch 12 | Train loss: 0.078908 | Val loss 0.069974
Epoch 13 | Train loss: 0.071287 | Val loss 0.091184
Epoch 14 | Train loss: 0.063468 | Val loss 0.069359
Epoch 15 | Train loss: 0.053189 | Val loss 0.056657
Epoch 16 | Train loss: 0.048219 | Val loss 0.053201
Epoch 17 | Train loss: 0.045864 | Val loss 0.046847
Epoch 18 | Train loss: 0.038355 | Val loss 0.045912
Epoch 19 | Train loss: 0.037792 | Val loss 0.039427
Epoch 20 | Train loss: 0.036408 | Val loss 0.033150
Epoch 21 | Train loss: 0.030287 | Val loss 0.033008
Epoch 22 | Train loss: 0.028561 | Val loss 0.033647
Epoch 23 | Train loss: 0.025631 | Val loss 0.032057
Epoch 24 | Train loss: 0.026862 | Val loss 0.031883
Epoch 25 | Train loss: 0.024851 | Val loss 0.033695
Epoch 26 | Train loss: 0.022077 | Val loss 0.027099
Epoch 27 | Train loss: 0.021219 | Val loss 0.030531
Epoch 28 | Train loss: 0.021120 | Val loss 0.028111
Epoch 29 | Train loss: 0.020262 | Val loss 0.024854
Epoch 30 | Train loss: 0.018058 | Val loss 0.023634
Epoch 31 | Train loss: 0.018963 | Val loss 0.031983
Epoch 32 | Train loss: 0.020428 | Val loss 0.024251
Epoch 33 | Train loss: 0.018113 | Val loss 0.024903
Epoch 34 | Train loss: 0.017480 | Val loss 0.023801
Epoch 35 | Train loss: 0.018471 | Val loss 0.025134
Epoch 36 | Train loss: 0.020613 | Val loss 0.024048
Epoch 37 | Train loss: 0.016554 | Val loss 0.022584
Epoch 38 | Train loss: 0.016326 | Val loss 0.021567
Epoch 39 | Train loss: 0.015004 | Val loss 0.019877

Epoch 40 | Train loss: 0.014156 | Val loss 0.020686
Epoch 41 | Train loss: 0.014286 | Val loss 0.020712
Epoch 42 | Train loss: 0.013818 | Val loss 0.021852
Epoch 43 | Train loss: 0.013485 | Val loss 0.021951
Epoch 44 | Train loss: 0.014023 | Val loss 0.022191
Epoch 45 | Train loss: 0.014137 | Val loss 0.020439
Epoch 46 | Train loss: 0.012959 | Val loss 0.021708
Epoch 47 | Train loss: 0.013028 | Val loss 0.020914
Epoch 48 | Train loss: 0.012803 | Val loss 0.020734
Epoch 49 | Train loss: 0.012217 | Val loss 0.020275
Epoch 50 | Train loss: 0.012005 | Val loss 0.020857

Training model: hidden_size=32, dataset=small

Epoch 1 | Train loss: 2.534132 | Val loss 2.202822
Epoch 2 | Train loss: 2.040287 | Val loss 1.884473
Epoch 3 | Train loss: 1.676998 | Val loss 1.423867
Epoch 4 | Train loss: 1.030204 | Val loss 0.693214
Epoch 5 | Train loss: 0.505794 | Val loss 0.434058
Epoch 6 | Train loss: 0.315031 | Val loss 0.336830
Epoch 7 | Train loss: 0.222192 | Val loss 0.211701
Epoch 8 | Train loss: 0.162393 | Val loss 0.313204
Epoch 9 | Train loss: 0.140160 | Val loss 0.171136
Epoch 10 | Train loss: 0.109359 | Val loss 0.128995
Epoch 11 | Train loss: 0.091312 | Val loss 0.132876
Epoch 12 | Train loss: 0.070537 | Val loss 0.140828
Epoch 13 | Train loss: 0.070850 | Val loss 0.110154
Epoch 14 | Train loss: 0.057708 | Val loss 0.098151
Epoch 15 | Train loss: 0.050758 | Val loss 0.096220
Epoch 16 | Train loss: 0.047984 | Val loss 0.088337
Epoch 17 | Train loss: 0.042278 | Val loss 0.084578
Epoch 18 | Train loss: 0.034694 | Val loss 0.077965
Epoch 19 | Train loss: 0.036182 | Val loss 0.081657
Epoch 20 | Train loss: 0.029975 | Val loss 0.073268

Epoch 21 | Train loss: 0.021376 | Val loss 0.073403
Epoch 22 | Train loss: 0.020821 | Val loss 0.072154
Epoch 23 | Train loss: 0.016152 | Val loss 0.075373
Epoch 24 | Train loss: 0.018267 | Val loss 0.071621
Epoch 25 | Train loss: 0.019921 | Val loss 0.077843
Epoch 26 | Train loss: 0.014957 | Val loss 0.087577
Epoch 27 | Train loss: 0.013425 | Val loss 0.076072
Epoch 28 | Train loss: 0.013746 | Val loss 0.072720
Epoch 29 | Train loss: 0.007288 | Val loss 0.073798
Epoch 30 | Train loss: 0.006029 | Val loss 0.073636
Epoch 31 | Train loss: 0.005847 | Val loss 0.075163
Epoch 32 | Train loss: 0.007363 | Val loss 0.081404
Epoch 33 | Train loss: 0.004994 | Val loss 0.075820
Epoch 34 | Train loss: 0.004612 | Val loss 0.074260
Epoch 35 | Train loss: 0.005923 | Val loss 0.076428
Epoch 36 | Train loss: 0.002694 | Val loss 0.082607
Epoch 37 | Train loss: 0.001809 | Val loss 0.085256
Epoch 38 | Train loss: 0.001268 | Val loss 0.082647
Epoch 39 | Train loss: 0.000945 | Val loss 0.086555
Epoch 40 | Train loss: 0.002492 | Val loss 0.098015
Epoch 41 | Train loss: 0.003225 | Val loss 0.088119
Epoch 42 | Train loss: 0.003495 | Val loss 0.092087
Epoch 43 | Train loss: 0.001834 | Val loss 0.079149
Epoch 44 | Train loss: 0.001305 | Val loss 0.083268
Epoch 45 | Train loss: 0.002002 | Val loss 0.080988
Epoch 46 | Train loss: 0.001055 | Val loss 0.080044
Epoch 47 | Train loss: 0.000615 | Val loss 0.087413
Epoch 48 | Train loss: 0.000643 | Val loss 0.091397
Epoch 49 | Train loss: 0.002301 | Val loss 0.085572
Epoch 50 | Train loss: 0.001455 | Val loss 0.083474

Training model: hidden_size=32, dataset=large

Epoch 1 | Train loss: 2.481611 | Val loss 2.149148

Epoch 2 | Train loss: 1.990227 | Val loss 1.887835
Epoch 3 | Train loss: 1.674761 | Val loss 1.382212
Epoch 4 | Train loss: 0.924879 | Val loss 0.570119
Epoch 5 | Train loss: 0.300984 | Val loss 0.163662
Epoch 6 | Train loss: 0.118117 | Val loss 0.074888
Epoch 7 | Train loss: 0.066509 | Val loss 0.057747
Epoch 8 | Train loss: 0.047386 | Val loss 0.072368
Epoch 9 | Train loss: 0.035741 | Val loss 0.033788
Epoch 10 | Train loss: 0.023924 | Val loss 0.028218
Epoch 11 | Train loss: 0.023837 | Val loss 0.037838
Epoch 12 | Train loss: 0.024454 | Val loss 0.026293
Epoch 13 | Train loss: 0.018297 | Val loss 0.019864
Epoch 14 | Train loss: 0.015548 | Val loss 0.029329
Epoch 15 | Train loss: 0.016352 | Val loss 0.019023
Epoch 16 | Train loss: 0.011057 | Val loss 0.014697
Epoch 17 | Train loss: 0.011341 | Val loss 0.031241
Epoch 18 | Train loss: 0.012270 | Val loss 0.015738
Epoch 19 | Train loss: 0.009251 | Val loss 0.018280
Epoch 20 | Train loss: 0.010412 | Val loss 0.017506
Epoch 21 | Train loss: 0.008109 | Val loss 0.014568
Epoch 22 | Train loss: 0.006628 | Val loss 0.016116
Epoch 23 | Train loss: 0.006321 | Val loss 0.012630
Epoch 24 | Train loss: 0.005642 | Val loss 0.013718
Epoch 25 | Train loss: 0.005854 | Val loss 0.016050
Epoch 26 | Train loss: 0.009145 | Val loss 0.027905
Epoch 27 | Train loss: 0.007004 | Val loss 0.013248
Epoch 28 | Train loss: 0.004443 | Val loss 0.012105
Epoch 29 | Train loss: 0.004736 | Val loss 0.012763
Epoch 30 | Train loss: 0.004968 | Val loss 0.014545
Epoch 31 | Train loss: 0.003254 | Val loss 0.014529
Epoch 32 | Train loss: 0.004543 | Val loss 0.020029
Epoch 33 | Train loss: 0.005736 | Val loss 0.012785

```
Epoch 34 | Train loss: 0.002915 | Val loss 0.013303
Epoch 35 | Train loss: 0.002372 | Val loss 0.012196
Epoch 36 | Train loss: 0.001586 | Val loss 0.010915
Epoch 37 | Train loss: 0.001310 | Val loss 0.012157
Epoch 38 | Train loss: 0.001190 | Val loss 0.012067
Epoch 39 | Train loss: 0.001541 | Val loss 0.012031
Epoch 40 | Train loss: 0.001078 | Val loss 0.011897
Epoch 41 | Train loss: 0.001029 | Val loss 0.012724
Epoch 42 | Train loss: 0.000964 | Val loss 0.013311
Epoch 43 | Train loss: 0.000846 | Val loss 0.013120
Epoch 44 | Train loss: 0.000769 | Val loss 0.012482
Epoch 45 | Train loss: 0.000667 | Val loss 0.012640
Epoch 46 | Train loss: 0.001231 | Val loss 0.013686
Epoch 47 | Train loss: 0.002305 | Val loss 0.014740
Epoch 48 | Train loss: 0.001035 | Val loss 0.013421
Epoch 49 | Train loss: 0.001532 | Val loss 0.012713
Epoch 50 | Train loss: 0.001100 | Val loss 0.012404
```

<Figure size 640x480 with 0 Axes>

<Figure size 640x480 with 0 Axes>

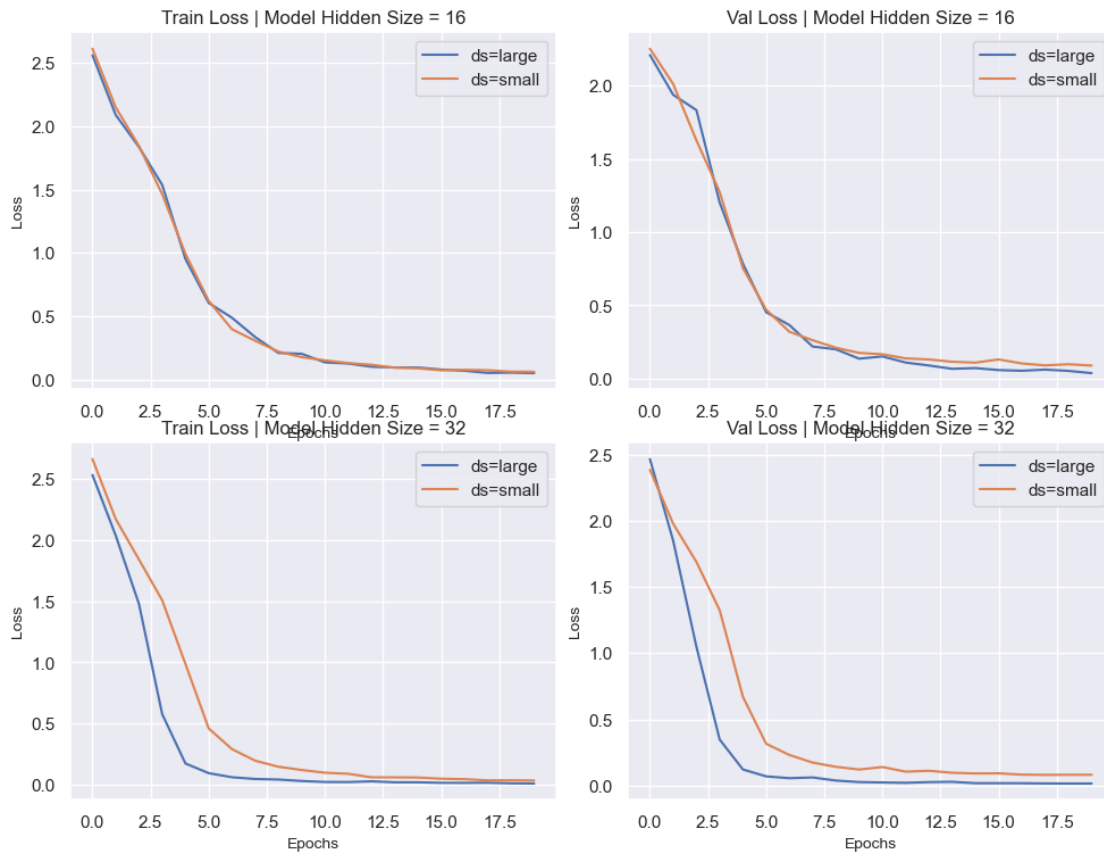
```
[ ]: fig1 = save_loss_comparison_by_dataset(all_results)
     fig2 = save_loss_comparison_by_hidden(all_results)
```

<Figure size 640x480 with 0 Axes>

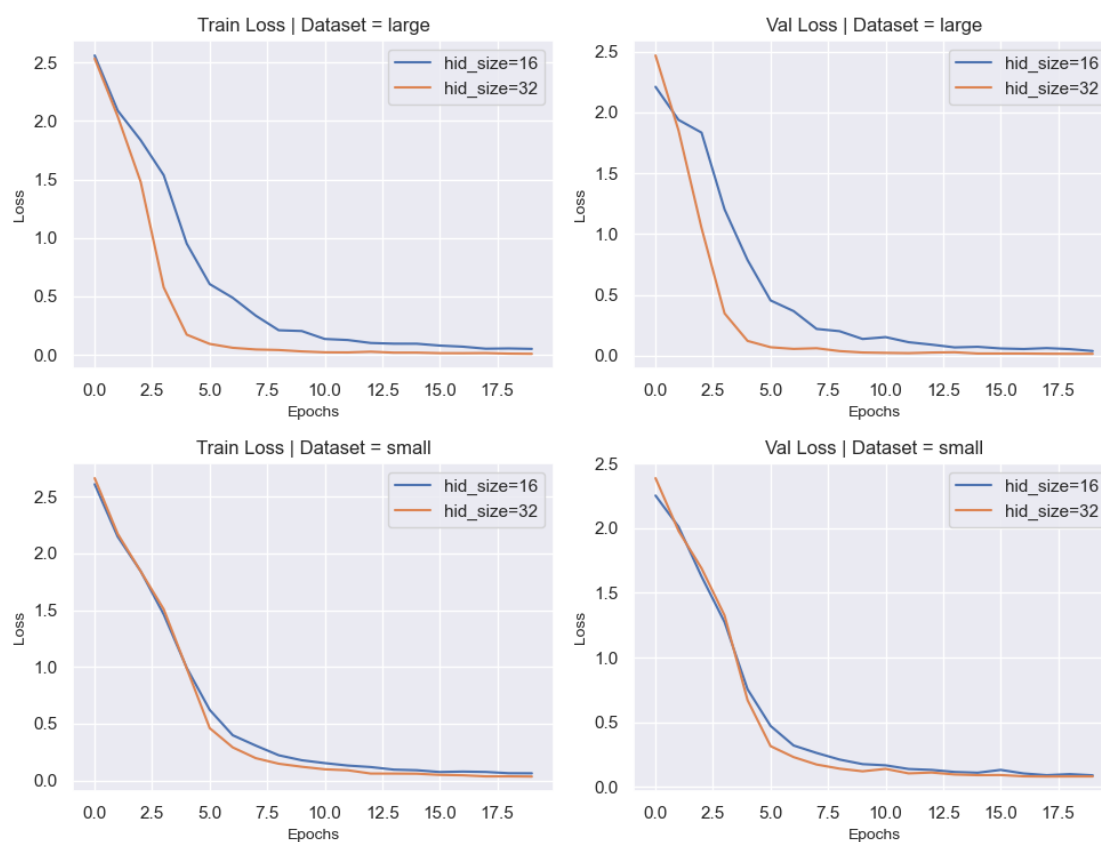
<Figure size 640x480 with 0 Axes>

```
[ ]: display(fig1)
     display(fig2)
```

Performance by Dataset Size



Performance by Hidden State Size



1.3.1 Step 4: Analyzing results

1. The two graphs paint a similar picture: The larger models perform better, and larger datasets lead to better performance. Could this be due to the difference in mini-batch sizes used for different dataset sizes?
2. The smaller model doesn't realize the improvements in training loss from the larger dataset as much as the larger models do
3. The larger smaller model performs much better relative to the smaller model on the large dataset than on the small dataset. This is probably because it better utilizes the additional data to learn more complex patterns.

2 Part 2: Decoder-only Transformers and Multi-Head Attention

In this section, you'll train transformers that only use a decoder instead of a dual encoder-decoder architecture. You'll also implement and experiment with multi-head attention. Finally, you'll compare increasing parameter count with multi-head attention vs. increasing the parameter count with a larger hidden dimensionality.

2.1 Step 1: Implement the Decoder-Only Forward Method

```
[23]: class DecoderOnlyBlock(nn.Module):
    def __init__(self, hidden_size: int):
        super().__init__()
        self.norm1 = LayerNorm(hidden_size)
        self.norm2 = LayerNorm(hidden_size)
        self.attn = CausalAttention(hidden_size)
        self.mlp = MLP(hidden_size)
        self.to(device)
        self.filename = model_filename(hidden_size=hidden_size)

    def forward(self, x: torch.Tensor):
        context, attention = self.attn(self.norm1(x))
        x = x + context
        x = x + self.mlp(self.norm2(x))
        return x, attention

class DecoderTransformer(nn.Module):
    def __init__(self, vocab_size: int, hidden_size: int, num_layers: int):
        super().__init__()

        self.tok_emb = nn.Embedding(vocab_size, hidden_size)
        self.pos_emb = PositionalEncoding(hidden_size)

        self.blocks = nn.ModuleList(
            [DecoderOnlyBlock(hidden_size) for i in range(num_layers)]
        )
        self.head = nn.Linear(hidden_size, vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight
        # weight tie, the final layer reuses the weights of the embedding layers
        # this reduces the total number of parameters
        self.to(device)
        self.filename = model_filename(
            hidden_size=hidden_size, num_layers=num_layers,
            vocab_size=vocab_size
        )

    def forward(self, inputs: torch.Tensor):
        """Forward pass of the Transformer decoder.

        Arguments:
            inputs: Input token indexes across a batch for all the time step.
            (batch_size x decoder_seq_len)

        Returns:
```

```

        output: Un-normalized scores for each token in the vocabulary,
        ↪ across a batch for all the decoding time steps. (batch_size x
        ↪ decoder_seq_len x vocab_size)
        attentions: The stacked attention weights applied to the encoder,
        ↪ annotations (batch_size x encoder_seq_len x decoder_seq_len)
        """
        # -----
        # FILL THIS IN
        # -----
        # pos =
        # x =
        # self_attn = []

        # for block in self.blocks:
        #     ...
        #
        # unnormalized_scores =
        pos = torch.arange(0, inputs.shape[-1], device=inputs.device).long()
        x = self.tok_emb(inputs) + self.pos_emb(pos)
        self_attn = []
        for block in self.blocks:
            x, s = block(x)
            self_attn.append(s)
        unnormalized_scores = self.head(x)

        return unnormalized_scores, {"self_attn": self_attn}

```

```

[24]: decoder_ds_small = DecoderDataset(data_path, "pig_latin_small")
      decoder_ds_large = DecoderDataset(data_path, "pig_latin_large")

```

```

[25]: print("Dataset sample:")
      print("Input:", decoder_ds_small.to_token(decoder_ds_small[0][0].tolist()))
      print("Target:", decoder_ds_small.to_token(decoder_ds_small[0][1].tolist()))

```

Dataset sample:

Input: <SOS>attacks<EOP>attacksway

Target: attacks<EOP>attacksway<EOS>

2.2 Step 2: Format for decoder-only generation.

Implement the missing line in `decoder_generate` to create an input string that looks like:

<SOS> input sequence <EOP>

You should use the `dataset.start_idx`, `dataset.eop_idx` attributes and the `dataset.to_idx()` method.

```

[26]: def decoder_generate(
      input: str,

```



```

    model: DecoderTransformer,
    dataset: DecoderDataset,
    max_len: int = 20,
):
    gen_string = ""
    # idx = TODO
    idx = dataset.to_idx(input)
    idx = torch.tensor([idx]).to(device)

    for i in range(max_len):
        logits = model(idx)[0]

        token_idx = logits[0, -1:].argmax(dim=-1)
        idx = torch.cat([idx, token_idx.unsqueeze(0)], dim=1)

        if token_idx.item() == dataset.end_idx:
            break
        else:
            gen_string = gen_string + dataset.idx_to_token[token_idx.item()]
    return gen_string

def decoder_translate(
    sentence: str, model: DecoderTransformer, dataset: DecoderDataset
):
    words = sentence.split()
    return " ".join([decoder_generate(w, model, dataset) for w in words])

```

2.3 Step 3: Train the model

We'll now train a decoder only model and test it out on an example. Use the output of this to guide your answer for question 3.

```

[27]: # model = load_model_auto(
#     DecoderTransformer,
#     vocab_size=len(decoder_ds_small.idx_to_token),
#     hidden_size=16,
#     num_layers=4,
# )
model = DecoderTransformer(
    vocab_size=len(decoder_ds_small.idx_to_token), hidden_size=16, num_layers=4
)

```

```

[28]: optimizer = optim.AdamW(model.parameters(), lr=1e-2)
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

results = train(

```

```

    model=model,
    dataset=decoder_ds_small,
    num_epochs=NUM_EPOCHS,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    batch_size=32,
)
save_model_auto(model)

```

```

Epoch 1 | Train loss: 3.083981 | Val loss 1.958903
Epoch 2 | Train loss: 1.759495 | Val loss 1.674273
Epoch 3 | Train loss: 1.551638 | Val loss 1.523412
Epoch 4 | Train loss: 1.394732 | Val loss 1.414328
Epoch 5 | Train loss: 1.281753 | Val loss 1.360562
Epoch 6 | Train loss: 1.182483 | Val loss 1.203522
Epoch 7 | Train loss: 1.066684 | Val loss 1.140728
Epoch 8 | Train loss: 0.975566 | Val loss 1.085813
Epoch 9 | Train loss: 0.891694 | Val loss 0.942689
Epoch 10 | Train loss: 0.815830 | Val loss 0.876949
Epoch 11 | Train loss: 0.748665 | Val loss 0.834969
Epoch 12 | Train loss: 0.689414 | Val loss 0.820004
Epoch 13 | Train loss: 0.635779 | Val loss 0.740955
Epoch 14 | Train loss: 0.587867 | Val loss 0.681685
Epoch 15 | Train loss: 0.539231 | Val loss 0.641578
Epoch 16 | Train loss: 0.503267 | Val loss 0.591732
Epoch 17 | Train loss: 0.472523 | Val loss 0.607404
Epoch 18 | Train loss: 0.444592 | Val loss 0.523673
Epoch 19 | Train loss: 0.423358 | Val loss 0.496591
Epoch 20 | Train loss: 0.389064 | Val loss 0.474809
Epoch 21 | Train loss: 0.368550 | Val loss 0.486579
Epoch 22 | Train loss: 0.353606 | Val loss 0.439805
Epoch 23 | Train loss: 0.334639 | Val loss 0.425623
Epoch 24 | Train loss: 0.312740 | Val loss 0.397954
Epoch 25 | Train loss: 0.297095 | Val loss 0.385238

```

```
Epoch 26 | Train loss: 0.278007 | Val loss 0.363985
Epoch 27 | Train loss: 0.258414 | Val loss 0.345033
Epoch 28 | Train loss: 0.246662 | Val loss 0.337073
Epoch 29 | Train loss: 0.227456 | Val loss 0.318279
Epoch 30 | Train loss: 0.220520 | Val loss 0.322890
Epoch 31 | Train loss: 0.203099 | Val loss 0.315370
Epoch 32 | Train loss: 0.195046 | Val loss 0.291731
Epoch 33 | Train loss: 0.184268 | Val loss 0.328856
Epoch 34 | Train loss: 0.174972 | Val loss 0.279499
Epoch 35 | Train loss: 0.167175 | Val loss 0.273124
Epoch 36 | Train loss: 0.164153 | Val loss 0.274936
Epoch 37 | Train loss: 0.154551 | Val loss 0.256587
Epoch 38 | Train loss: 0.146067 | Val loss 0.248466
Epoch 39 | Train loss: 0.138013 | Val loss 0.252847
Epoch 40 | Train loss: 0.142586 | Val loss 0.263248
Epoch 41 | Train loss: 0.128114 | Val loss 0.246953
Epoch 42 | Train loss: 0.125082 | Val loss 0.232297
Epoch 43 | Train loss: 0.120724 | Val loss 0.246692
Epoch 44 | Train loss: 0.119419 | Val loss 0.235251
Epoch 45 | Train loss: 0.113544 | Val loss 0.233631
Epoch 46 | Train loss: 0.111224 | Val loss 0.227365
Epoch 47 | Train loss: 0.104624 | Val loss 0.224217
Epoch 48 | Train loss: 0.100682 | Val loss 0.226935
Epoch 49 | Train loss: 0.099131 | Val loss 0.221974
Epoch 50 | Train loss: 0.093716 | Val loss 0.231623
```

```
[28]: PosixPath('/Users/Akseldkw/coding/Columbia/COMS4776-
Data/data/homework/HW2/DecoderTransformer/50_epochs/Model_vocab30_hid16_layers4.
pt')
```

```
[29]: decoder_translate("the air conditioning is working", model, decoder_ds_small)
```

```
[29]: 'a-aa-a-aaay-ay-aaaaa aaaaaaaaaaaaaaaaaaaaaa way-ionagay-aaay-ay-
iiiiiiiiiiiiiiiiiii <SOS><SOS><SOS>--khwhwkvvvvvrrr'
```

2.4 Step 4: Implement Multi-Head Attention

In this next step, you'll implement multi-head attention. In this setting, we'll have the dimensionality for each of the heads be $(\text{total hidden dimensionality})/(\text{number of heads})$. We'll use a projection head to combine the outputs of these different attention heads after concatenating their outputs together (which can also be achieved using tensor reshaping).

```
[30]: # TO DO:

def multihead_scaled_dot_attention(
    q: torch.Tensor, k: torch.Tensor, v: torch.Tensor, mask: torch.Tensor | None = None
):
    """Compute multi-head dot product attention.
    q : shape (batch, num_heads, k, hidden_size)
    k : shape (batch, num_heads, seq_len, hidden_size)
    v : shape (batch, num_heads, seq_len, hidden_size)

    returns:
        context : shape (batch, num_heads, k, hidden_size)
        attention : shape (batch, num_heads, k, seq_len)
    """
    # Hint: if you implemented scaled_dot_attention well, the code for this
    # should be the exact same.

    # -----
    # FILL THIS IN
    # -----
    # unnorm_attn =
    # masked_unnorm_attn =
    # attention =
    # context =
    unnorm_attn = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(q.size(-1))
    if mask is not None:
        unnorm_attn = unnorm_attn.masked_fill(mask == 0, float("-inf"))
    attention = F.softmax(unnorm_attn, dim=-1)
    context = torch.matmul(attention, v)
    return context, attention
```

```
[31]: class MultiheadCausalAttention(Attention):
    proj: nn.Linear

    def __init__(self, hidden_size: int, num_heads: int):
        super().__init__(hidden_size)
        # self.Q, self.K, self.V already defined as nn.Linear(hidden_size,
        # hidden_size)
        self.num_heads = num_heads
```

```

# -----
# FILL THIS IN
# -----
# if self.num_heads > 1:
#     self.proj = nn.Linear( , , bias=False)
if self.num_heads > 1:
    self.proj = nn.Linear(hidden_size, hidden_size, bias=False)
self.to(device)
self.filename = model_filename(hidden_size=hidden_size,
↪num_heads=num_heads)

def forward(self, x: torch.Tensor):
    """Compute causal multi-head dot product attention from an input
↪sequence.
        x : shape ( batch, seq_len, hidden_size)
        k : shape (batch, num_heads, seq_len, hidden_size)
        v : shape (batch, num_heads, seq_len, hidden_size)

    returns:
        context : shape ( batch, seq_len, hidden_size)
        attention : shape (batch, num_heads, k, seq_len)
    """
    # -----
    # FILL THIS IN
    # -----
    # unnorm_attn =
    # masked_unnorm_attn =
    # attention =
    # context =

    # B, T, C = x.size()
    # q = self.Q(x).view( , , , ).transpose(1, 2)
    # k = self.K(x).view( , , , ).transpose(1, 2)
    # v = self.V(x).view( , , , ).transpose(1, 2)

    # mask =

    # context, attention = multihead_scaled_dot_attention(q, k, v, mask)

    # Hint: you may find the .transpose(), .contiguous(), and .view()
    # methods helpful for creating the reshaped context.

    # reshaped_context =

    # Only perform a projection if the number of heads is more than 1.
    B, T, C = x.size()

```

```

        q = self.Q(x).view(B, T, self.num_heads, C // self.num_heads).
↳transpose(1, 2)
        k = self.K(x).view(B, T, self.num_heads, C // self.num_heads).
↳transpose(1, 2)
        v = self.V(x).view(B, T, self.num_heads, C // self.num_heads).
↳transpose(1, 2)
        mask = torch.tril(torch.ones((T, T), device=x.device)).bool()
        context, attention = multihead_scaled_dot_attention(q, k, v, mask)
        reshaped_context = context.transpose(1, 2).contiguous().view(B, T, C)
        if self.num_heads > 1:

            return self.proj(reshaped_context), attention

        else:

            return reshaped_context, attention

```

```

[32]: class MultiheadDecoderBlock(DecoderOnlyBlock):
        def __init__(self, hidden_size: int, num_heads: int):
            super().__init__(hidden_size)
            self.attn = MultiheadCausalAttention(hidden_size, num_heads)
            self.to(device)
            self.filename = model_filename(hidden_size=hidden_size,
↳num_heads=num_heads)

class MultiheadDecoderTransformer(DecoderTransformer):
    def __init__(
        self, vocab_size: int, hidden_size: int, num_layers: int, num_heads: int
    ):
        super().__init__(vocab_size, hidden_size, num_layers)
        self.num_heads = num_heads
        self.num_layers = num_layers
        self.vocab_size = vocab_size
        self.hidden_size = hidden_size

        self.blocks = nn.ModuleList(
            [MultiheadDecoderBlock(hidden_size, num_heads) for i in
↳range(num_layers)]
        )
        self.to(device)
        self.filename = model_filename(
            hidden_size=hidden_size,
            num_layers=num_layers,
            vocab_size=vocab_size,
            num_heads=num_heads,
        )

```

2.5 Step 5: Compare Parameter Usage Methods

Here, you'll train 5 models with varying numbers of attention heads and hidden dimensionalities. You'll compare their parameter efficiency. Start by implementing the `calc_parameters_decoder` method, and then run the code to train the models.

```
[33]: def calc_parameters_decoder(model: MultiheadDecoderTransformer) -> int:
    # -----
    # FILL THIS IN
    # -----
    # params = TODO # embedding
    # params += TODO # QKV linear projections
    # if model.num_heads > 1:
    #     params += TODO # attention linear projection
    # params += TODO # MLP linear
    # params += TODO # layer norms Hint: LayerNorm(x) uses x parameters

    # Hint: Read the code carefully:
    # params += TODO # head
    params = model.vocab_size * model.hidden_size # embedding
    params += 3 * (model.hidden_size * model.hidden_size) # QKV linear
    ↪projections
    if model.num_heads > 1:
        params += model.hidden_size * model.hidden_size # attention linear
    ↪projection
        params += 2 * (
            4 * model.hidden_size * model.hidden_size
        ) # MLP linear (2 per block)
    params += 2 * 2 * model.hidden_size # layer norms (2 per block)

    return params
```

```
[34]: import matplotlib.pyplot as plt
import numpy as np

configs = [
    (16, 4, 4, 64), # (hidden_size, num_heads, num_layers, batch_size)
    (16, 2, 4, 64),
    (16, 1, 4, 64),
    (24, 1, 4, 64),
    (32, 1, 4, 64),
]

all_results = {}

for hidden_size, num_heads, num_layers, batch_size in configs:
    print(
```

```

        f"\nTraining model: hidden_size={hidden_size}, num_heads={num_heads},\n
        num_layers={num_layers}"
    )
    # model = load_model_auto(
    #     MultiheadDecoderTransformer,
    #     vocab_size=len(decoder_ds_large.idx_to_token),
    #     hidden_size=hidden_size,
    #     num_layers=num_layers,
    #     num_heads=num_heads,
    # )
    model = MultiheadDecoderTransformer(
        vocab_size=len(decoder_ds_large.idx_to_token),
        hidden_size=hidden_size,
        num_layers=num_layers,
        num_heads=num_heads,
    )

    optimizer = optim.AdamW(model.parameters(), lr=5e-3)
    scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

    results = train(
        model=model,
        dataset=decoder_ds_large,
        num_epochs=NUM_EPOCHS,
        optimizer=optimizer,
        scheduler=scheduler,
        device=device,
        batch_size=batch_size,
    )
    save_model_auto(model)

    parameters = calc_parameters_decoder(model)
    x_points = np.linspace(0, len(results["val_loss"]),
        len(results["val_loss"]))
    plt.plot(
        x_points,
        results["val_loss"],
        label="heads = {}, dim={}, params={}".format(
            num_heads, hidden_size, parameters
        ),
    )

plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
plt.title("Compute Optimal Models")
plt.grid()
plt.legend()

```



```
plt.show()
```

Training model: hidden_size=16, num_heads=4, num_layers=4

Epoch 1	Train loss: 2.217859	Val loss 1.311012
Epoch 2	Train loss: 0.999941	Val loss 0.728174
Epoch 3	Train loss: 0.509989	Val loss 0.401857
Epoch 4	Train loss: 0.290280	Val loss 0.247309
Epoch 5	Train loss: 0.189157	Val loss 0.166002
Epoch 6	Train loss: 0.138843	Val loss 0.116156
Epoch 7	Train loss: 0.112719	Val loss 0.096712
Epoch 8	Train loss: 0.090081	Val loss 0.064683
Epoch 9	Train loss: 0.081399	Val loss 0.084622
Epoch 10	Train loss: 0.068543	Val loss 0.067254
Epoch 11	Train loss: 0.060570	Val loss 0.049043
Epoch 12	Train loss: 0.050668	Val loss 0.052342
Epoch 13	Train loss: 0.048192	Val loss 0.070524
Epoch 14	Train loss: 0.044426	Val loss 0.046999
Epoch 15	Train loss: 0.042160	Val loss 0.043765
Epoch 16	Train loss: 0.037634	Val loss 0.032053
Epoch 17	Train loss: 0.033423	Val loss 0.032092
Epoch 18	Train loss: 0.028195	Val loss 0.026579
Epoch 19	Train loss: 0.027443	Val loss 0.036336
Epoch 20	Train loss: 0.026821	Val loss 0.032178
Epoch 21	Train loss: 0.024219	Val loss 0.026176
Epoch 22	Train loss: 0.021764	Val loss 0.023791
Epoch 23	Train loss: 0.021640	Val loss 0.022420
Epoch 24	Train loss: 0.018936	Val loss 0.034380
Epoch 25	Train loss: 0.020045	Val loss 0.018979
Epoch 26	Train loss: 0.017095	Val loss 0.030751
Epoch 27	Train loss: 0.017351	Val loss 0.019186
Epoch 28	Train loss: 0.016086	Val loss 0.023933
Epoch 29	Train loss: 0.014720	Val loss 0.017392

Epoch 30 | Train loss: 0.014028 | Val loss 0.020440
Epoch 31 | Train loss: 0.013050 | Val loss 0.017941
Epoch 32 | Train loss: 0.012724 | Val loss 0.015246
Epoch 33 | Train loss: 0.011707 | Val loss 0.014713
Epoch 34 | Train loss: 0.011409 | Val loss 0.013151
Epoch 35 | Train loss: 0.011011 | Val loss 0.014541
Epoch 36 | Train loss: 0.009988 | Val loss 0.013832
Epoch 37 | Train loss: 0.009707 | Val loss 0.015345
Epoch 38 | Train loss: 0.009245 | Val loss 0.012526
Epoch 39 | Train loss: 0.009156 | Val loss 0.013217
Epoch 40 | Train loss: 0.008271 | Val loss 0.013649
Epoch 41 | Train loss: 0.007718 | Val loss 0.012658
Epoch 42 | Train loss: 0.008290 | Val loss 0.013652
Epoch 43 | Train loss: 0.008108 | Val loss 0.012041
Epoch 44 | Train loss: 0.007211 | Val loss 0.014533
Epoch 45 | Train loss: 0.006797 | Val loss 0.011933
Epoch 46 | Train loss: 0.006166 | Val loss 0.011747
Epoch 47 | Train loss: 0.006316 | Val loss 0.010793
Epoch 48 | Train loss: 0.006010 | Val loss 0.011145
Epoch 49 | Train loss: 0.005237 | Val loss 0.010884
Epoch 50 | Train loss: 0.005419 | Val loss 0.010492

Training model: hidden_size=16, num_heads=2, num_layers=4

Epoch 1 | Train loss: 2.193548 | Val loss 1.336008
Epoch 2 | Train loss: 0.952102 | Val loss 0.699363
Epoch 3 | Train loss: 0.438443 | Val loss 0.305760
Epoch 4 | Train loss: 0.224735 | Val loss 0.183252
Epoch 5 | Train loss: 0.147191 | Val loss 0.097320
Epoch 6 | Train loss: 0.109524 | Val loss 0.094979
Epoch 7 | Train loss: 0.087570 | Val loss 0.084928
Epoch 8 | Train loss: 0.072226 | Val loss 0.072530
Epoch 9 | Train loss: 0.063869 | Val loss 0.058354
Epoch 10 | Train loss: 0.057116 | Val loss 0.063123

Epoch 11		Train loss: 0.046083		Val loss 0.062242
Epoch 12		Train loss: 0.042366		Val loss 0.033092
Epoch 13		Train loss: 0.043618		Val loss 0.057605
Epoch 14		Train loss: 0.033819		Val loss 0.041529
Epoch 15		Train loss: 0.032659		Val loss 0.029760
Epoch 16		Train loss: 0.030224		Val loss 0.026934
Epoch 17		Train loss: 0.026884		Val loss 0.022594
Epoch 18		Train loss: 0.024217		Val loss 0.025009
Epoch 19		Train loss: 0.022695		Val loss 0.027609
Epoch 20		Train loss: 0.021679		Val loss 0.030969
Epoch 21		Train loss: 0.021131		Val loss 0.020211
Epoch 22		Train loss: 0.019476		Val loss 0.023150
Epoch 23		Train loss: 0.017282		Val loss 0.018155
Epoch 24		Train loss: 0.016623		Val loss 0.019543
Epoch 25		Train loss: 0.015584		Val loss 0.018688
Epoch 26		Train loss: 0.014108		Val loss 0.015478
Epoch 27		Train loss: 0.014277		Val loss 0.013898
Epoch 28		Train loss: 0.013312		Val loss 0.015661
Epoch 29		Train loss: 0.012079		Val loss 0.013960
Epoch 30		Train loss: 0.011681		Val loss 0.012730
Epoch 31		Train loss: 0.011369		Val loss 0.011365
Epoch 32		Train loss: 0.010030		Val loss 0.011953
Epoch 33		Train loss: 0.009350		Val loss 0.014140
Epoch 34		Train loss: 0.009967		Val loss 0.015518
Epoch 35		Train loss: 0.009915		Val loss 0.013232
Epoch 36		Train loss: 0.008303		Val loss 0.010950
Epoch 37		Train loss: 0.008185		Val loss 0.013059
Epoch 38		Train loss: 0.006993		Val loss 0.010048
Epoch 39		Train loss: 0.006731		Val loss 0.012519
Epoch 40		Train loss: 0.006626		Val loss 0.013221
Epoch 41		Train loss: 0.006079		Val loss 0.010544
Epoch 42		Train loss: 0.006212		Val loss 0.009564

Epoch 43 | Train loss: 0.005263 | Val loss 0.012078
Epoch 44 | Train loss: 0.005358 | Val loss 0.013896
Epoch 45 | Train loss: 0.005680 | Val loss 0.010978
Epoch 46 | Train loss: 0.004897 | Val loss 0.008749
Epoch 47 | Train loss: 0.004353 | Val loss 0.010620
Epoch 48 | Train loss: 0.004589 | Val loss 0.010269
Epoch 49 | Train loss: 0.004391 | Val loss 0.009851
Epoch 50 | Train loss: 0.003922 | Val loss 0.009770

Training model: hidden_size=16, num_heads=1, num_layers=4

Epoch 1 | Train loss: 2.214834 | Val loss 1.311426
Epoch 2 | Train loss: 0.946759 | Val loss 0.643802
Epoch 3 | Train loss: 0.472769 | Val loss 0.315044
Epoch 4 | Train loss: 0.236641 | Val loss 0.193517
Epoch 5 | Train loss: 0.168007 | Val loss 0.128391
Epoch 6 | Train loss: 0.126884 | Val loss 0.127566
Epoch 7 | Train loss: 0.102108 | Val loss 0.124840
Epoch 8 | Train loss: 0.101662 | Val loss 0.078720
Epoch 9 | Train loss: 0.076352 | Val loss 0.061762
Epoch 10 | Train loss: 0.076840 | Val loss 0.082938
Epoch 11 | Train loss: 0.065312 | Val loss 0.070063
Epoch 12 | Train loss: 0.057403 | Val loss 0.095613
Epoch 13 | Train loss: 0.055159 | Val loss 0.045623
Epoch 14 | Train loss: 0.045982 | Val loss 0.047561
Epoch 15 | Train loss: 0.047513 | Val loss 0.043306
Epoch 16 | Train loss: 0.041881 | Val loss 0.042814
Epoch 17 | Train loss: 0.040380 | Val loss 0.042269
Epoch 18 | Train loss: 0.034438 | Val loss 0.034436
Epoch 19 | Train loss: 0.035285 | Val loss 0.038176
Epoch 20 | Train loss: 0.032020 | Val loss 0.032373
Epoch 21 | Train loss: 0.030694 | Val loss 0.067918
Epoch 22 | Train loss: 0.030860 | Val loss 0.036165
Epoch 23 | Train loss: 0.027141 | Val loss 0.027088

Epoch 24 | Train loss: 0.024648 | Val loss 0.027941
Epoch 25 | Train loss: 0.025429 | Val loss 0.024295
Epoch 26 | Train loss: 0.025538 | Val loss 0.020481
Epoch 27 | Train loss: 0.022775 | Val loss 0.022640
Epoch 28 | Train loss: 0.022834 | Val loss 0.023113
Epoch 29 | Train loss: 0.020156 | Val loss 0.022469
Epoch 30 | Train loss: 0.021173 | Val loss 0.020743
Epoch 31 | Train loss: 0.018626 | Val loss 0.020026
Epoch 32 | Train loss: 0.017940 | Val loss 0.020489
Epoch 33 | Train loss: 0.018758 | Val loss 0.020647
Epoch 34 | Train loss: 0.016183 | Val loss 0.019033
Epoch 35 | Train loss: 0.016115 | Val loss 0.017481
Epoch 36 | Train loss: 0.015469 | Val loss 0.019617
Epoch 37 | Train loss: 0.014208 | Val loss 0.018656
Epoch 38 | Train loss: 0.014734 | Val loss 0.019352
Epoch 39 | Train loss: 0.014323 | Val loss 0.021719
Epoch 40 | Train loss: 0.013057 | Val loss 0.019295
Epoch 41 | Train loss: 0.012995 | Val loss 0.017095
Epoch 42 | Train loss: 0.012082 | Val loss 0.015972
Epoch 43 | Train loss: 0.012467 | Val loss 0.014286
Epoch 44 | Train loss: 0.011200 | Val loss 0.017129
Epoch 45 | Train loss: 0.011014 | Val loss 0.016068
Epoch 46 | Train loss: 0.010301 | Val loss 0.015933
Epoch 47 | Train loss: 0.009930 | Val loss 0.013743
Epoch 48 | Train loss: 0.010464 | Val loss 0.014681
Epoch 49 | Train loss: 0.009327 | Val loss 0.013190
Epoch 50 | Train loss: 0.009528 | Val loss 0.015138

Training model: hidden_size=24, num_heads=1, num_layers=4

Epoch 1 | Train loss: 1.905115 | Val loss 0.527164
Epoch 2 | Train loss: 0.304629 | Val loss 0.199109
Epoch 3 | Train loss: 0.155150 | Val loss 0.134811
Epoch 4 | Train loss: 0.107475 | Val loss 0.102890

Epoch 5 | Train loss: 0.083913 | Val loss 0.083515
Epoch 6 | Train loss: 0.069090 | Val loss 0.053473
Epoch 7 | Train loss: 0.051710 | Val loss 0.044656
Epoch 8 | Train loss: 0.047406 | Val loss 0.044869
Epoch 9 | Train loss: 0.037329 | Val loss 0.045176
Epoch 10 | Train loss: 0.037202 | Val loss 0.028370
Epoch 11 | Train loss: 0.031550 | Val loss 0.029944
Epoch 12 | Train loss: 0.028248 | Val loss 0.021308
Epoch 13 | Train loss: 0.021290 | Val loss 0.024362
Epoch 14 | Train loss: 0.022983 | Val loss 0.025149
Epoch 15 | Train loss: 0.019897 | Val loss 0.024110
Epoch 16 | Train loss: 0.018017 | Val loss 0.018745
Epoch 17 | Train loss: 0.015610 | Val loss 0.043727
Epoch 18 | Train loss: 0.015266 | Val loss 0.017976
Epoch 19 | Train loss: 0.013013 | Val loss 0.021405
Epoch 20 | Train loss: 0.013377 | Val loss 0.014909
Epoch 21 | Train loss: 0.012803 | Val loss 0.021042
Epoch 22 | Train loss: 0.010235 | Val loss 0.020033
Epoch 23 | Train loss: 0.010933 | Val loss 0.014283
Epoch 24 | Train loss: 0.009971 | Val loss 0.013869
Epoch 25 | Train loss: 0.006946 | Val loss 0.010670
Epoch 26 | Train loss: 0.007556 | Val loss 0.019069
Epoch 27 | Train loss: 0.008054 | Val loss 0.015872
Epoch 28 | Train loss: 0.006040 | Val loss 0.013538
Epoch 29 | Train loss: 0.007026 | Val loss 0.013527
Epoch 30 | Train loss: 0.004920 | Val loss 0.010787
Epoch 31 | Train loss: 0.004301 | Val loss 0.016198
Epoch 32 | Train loss: 0.004365 | Val loss 0.009749
Epoch 33 | Train loss: 0.004850 | Val loss 0.017055
Epoch 34 | Train loss: 0.003830 | Val loss 0.007385
Epoch 35 | Train loss: 0.003804 | Val loss 0.009222
Epoch 36 | Train loss: 0.003311 | Val loss 0.008534

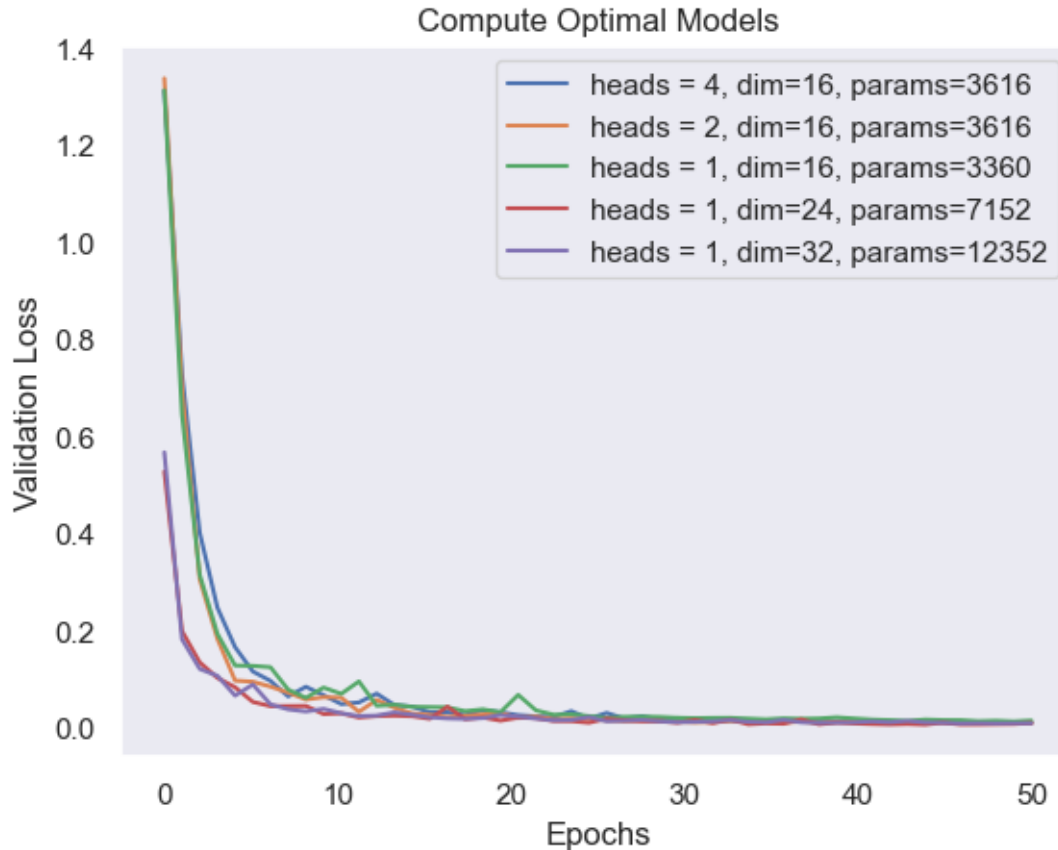
Epoch 37 | Train loss: 0.002862 | Val loss 0.017631
Epoch 38 | Train loss: 0.003224 | Val loss 0.007345
Epoch 39 | Train loss: 0.003320 | Val loss 0.010829
Epoch 40 | Train loss: 0.002735 | Val loss 0.008984
Epoch 41 | Train loss: 0.002915 | Val loss 0.007898
Epoch 42 | Train loss: 0.002289 | Val loss 0.007085
Epoch 43 | Train loss: 0.001374 | Val loss 0.008019
Epoch 44 | Train loss: 0.001707 | Val loss 0.006761
Epoch 45 | Train loss: 0.001589 | Val loss 0.010618
Epoch 46 | Train loss: 0.001769 | Val loss 0.007095
Epoch 47 | Train loss: 0.000958 | Val loss 0.007210
Epoch 48 | Train loss: 0.000994 | Val loss 0.007618
Epoch 49 | Train loss: 0.002238 | Val loss 0.007948
Epoch 50 | Train loss: 0.001596 | Val loss 0.009904

Training model: hidden_size=32, num_heads=1, num_layers=4

Epoch 1 | Train loss: 2.052315 | Val loss 0.566767
Epoch 2 | Train loss: 0.327843 | Val loss 0.182200
Epoch 3 | Train loss: 0.153887 | Val loss 0.121308
Epoch 4 | Train loss: 0.102960 | Val loss 0.107122
Epoch 5 | Train loss: 0.073346 | Val loss 0.066347
Epoch 6 | Train loss: 0.055902 | Val loss 0.089281
Epoch 7 | Train loss: 0.046244 | Val loss 0.048426
Epoch 8 | Train loss: 0.036429 | Val loss 0.038132
Epoch 9 | Train loss: 0.030927 | Val loss 0.033799
Epoch 10 | Train loss: 0.026460 | Val loss 0.039032
Epoch 11 | Train loss: 0.022794 | Val loss 0.030153
Epoch 12 | Train loss: 0.020793 | Val loss 0.024427
Epoch 13 | Train loss: 0.017922 | Val loss 0.025326
Epoch 14 | Train loss: 0.017352 | Val loss 0.031970
Epoch 15 | Train loss: 0.014529 | Val loss 0.027273
Epoch 16 | Train loss: 0.011579 | Val loss 0.021789
Epoch 17 | Train loss: 0.011108 | Val loss 0.019427

Epoch 18		Train loss: 0.011267		Val loss 0.018578
Epoch 19		Train loss: 0.010787		Val loss 0.020266
Epoch 20		Train loss: 0.009900		Val loss 0.026367
Epoch 21		Train loss: 0.007298		Val loss 0.023156
Epoch 22		Train loss: 0.007567		Val loss 0.019240
Epoch 23		Train loss: 0.005909		Val loss 0.015472
Epoch 24		Train loss: 0.005584		Val loss 0.016331
Epoch 25		Train loss: 0.005007		Val loss 0.022210
Epoch 26		Train loss: 0.006512		Val loss 0.013681
Epoch 27		Train loss: 0.004631		Val loss 0.014593
Epoch 28		Train loss: 0.004525		Val loss 0.016319
Epoch 29		Train loss: 0.002768		Val loss 0.013338
Epoch 30		Train loss: 0.001978		Val loss 0.011782
Epoch 31		Train loss: 0.002376		Val loss 0.012353
Epoch 32		Train loss: 0.002485		Val loss 0.013491
Epoch 33		Train loss: 0.004307		Val loss 0.016016
Epoch 34		Train loss: 0.002119		Val loss 0.012358
Epoch 35		Train loss: 0.001193		Val loss 0.011207
Epoch 36		Train loss: 0.002497		Val loss 0.016320
Epoch 37		Train loss: 0.001700		Val loss 0.011401
Epoch 38		Train loss: 0.000789		Val loss 0.010270
Epoch 39		Train loss: 0.000774		Val loss 0.010822
Epoch 40		Train loss: 0.001286		Val loss 0.011304
Epoch 41		Train loss: 0.001277		Val loss 0.013180
Epoch 42		Train loss: 0.001167		Val loss 0.011453
Epoch 43		Train loss: 0.002506		Val loss 0.014304
Epoch 44		Train loss: 0.001054		Val loss 0.011101
Epoch 45		Train loss: 0.000925		Val loss 0.010458
Epoch 46		Train loss: 0.000603		Val loss 0.009222
Epoch 47		Train loss: 0.000453		Val loss 0.009693
Epoch 48		Train loss: 0.000529		Val loss 0.009202
Epoch 49		Train loss: 0.000425		Val loss 0.009571

Epoch 50 | Train loss: 0.000388 | Val loss 0.009493



```
[35]: plt.clf()
```

<Figure size 640x480 with 0 Axes>

2.5.1 Step 4: Analysis of Results

It looks like all models trained for 50 epochs converged to a similar validation loss. However, it looks like the model with fewer heads started with much lower training loss and converged faster. Specifically, the 1 head model with 24 or 32 hidden units had the lowest validation loss at each epoch. Notably, the single head 16 hidden unit didn't perform as well, indicating that there is a minimum hidden size required to model the data well (and additionally demonstrated higher variance in training loss).

It would be interesting to see how these models performed on other (more complicated) datasets, and guess at whether the optimal model size would be indicative of the dataset complexity.

3 Part 3: IsoFLOP Profiling and Scaling Laws

In this final section, you'll compare different architectures based on their compute efficiency. You will also determine the optimal number of parameters and tokens for a fixed compute budget.

While this section does not require any additional code from you, **use the output of it to guide your written responses.**

```
[36]: from utils import (
        plot_scaling_law,
        plot_scaling_law_poly,
        interpolate,
        plot_flops_tokens,
        plot_isoflop,
        plot_flops_params,
    )

    import numpy as np
```

3.1 Step 1: Train models, Compare FLOPS vs Validation Loss

We'll inspect the compute efficiency of different models by plotting their validation loss against the number of FLOPs used in training up to that point.

```
[37]: from types_hw2 import TrainerHistDict
```

```
[38]: configs = [
        (40, 2, 4, 256), # (hidden_size, num_heads, num_layers, batch_size)
        (32, 2, 4, 128),
        (24, 2, 4, 64),
        (16, 1, 4, 64),
        (16, 1, 3, 64),
        (12, 1, 3, 64),
        (12, 1, 2, 64),
    ]

    all_results: dict[int, TrainerHistDict] = {}

    for hidden_size, num_heads, num_layers, batch_size in configs:
        print(
            f"\nTraining model: hidden_size={hidden_size}, num_heads={num_heads},
            ↪num_layers={num_layers}"
        )

        # model = load_model_auto(
        #     MultiheadDecoderTransformer,
        #     vocab_size=len(decoder_ds_large.idx_to_token),
        #     hidden_size=hidden_size,
```

```

#     num_layers=num_layers,
#     num_heads=num_heads,
# )
model = MultiheadDecoderTransformer(
    vocab_size=len(decoder_ds_large.idx_to_token),
    hidden_size=hidden_size,
    num_layers=num_layers,
    num_heads=num_heads,
)

optimizer = optim.AdamW(model.parameters(), lr=5e-3)
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

results = train(
    model=model,
    dataset=decoder_ds_large,
    num_epochs=NUM_EPOCHS,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    batch_size=batch_size,
)
save_model_auto(model)

parameters = calc_parameters_decoder(model)
all_results[parameters] = results

```

Training model: hidden_size=40, num_heads=2, num_layers=4

```

Epoch 1 | Train loss: 4.759319 | Val loss 1.367960
Epoch 2 | Train loss: 0.846089 | Val loss 0.382018
Epoch 3 | Train loss: 0.258866 | Val loss 0.294434
Epoch 4 | Train loss: 0.134634 | Val loss 0.088001
Epoch 5 | Train loss: 0.079121 | Val loss 0.088974
Epoch 6 | Train loss: 0.056373 | Val loss 0.043615
Epoch 7 | Train loss: 0.038149 | Val loss 0.038321
Epoch 8 | Train loss: 0.036324 | Val loss 0.029705
Epoch 9 | Train loss: 0.023225 | Val loss 0.024885
Epoch 10 | Train loss: 0.022647 | Val loss 0.033912
Epoch 11 | Train loss: 0.019451 | Val loss 0.026247
Epoch 12 | Train loss: 0.018572 | Val loss 0.021343

```

Epoch 13		Train loss: 0.009325		Val loss 0.014339
Epoch 14		Train loss: 0.010795		Val loss 0.023311
Epoch 15		Train loss: 0.009605		Val loss 0.017701
Epoch 16		Train loss: 0.009718		Val loss 0.014133
Epoch 17		Train loss: 0.005255		Val loss 0.013638
Epoch 18		Train loss: 0.003406		Val loss 0.010949
Epoch 19		Train loss: 0.002110		Val loss 0.011267
Epoch 20		Train loss: 0.001825		Val loss 0.011000
Epoch 21		Train loss: 0.001775		Val loss 0.013193
Epoch 22		Train loss: 0.002547		Val loss 0.014857
Epoch 23		Train loss: 0.005608		Val loss 0.015327
Epoch 24		Train loss: 0.004912		Val loss 0.011155
Epoch 25		Train loss: 0.002438		Val loss 0.010049
Epoch 26		Train loss: 0.001303		Val loss 0.009112
Epoch 27		Train loss: 0.001074		Val loss 0.008856
Epoch 28		Train loss: 0.000780		Val loss 0.009320
Epoch 29		Train loss: 0.000723		Val loss 0.009321
Epoch 30		Train loss: 0.000689		Val loss 0.010133
Epoch 31		Train loss: 0.000603		Val loss 0.009543
Epoch 32		Train loss: 0.000563		Val loss 0.009763
Epoch 33		Train loss: 0.000583		Val loss 0.009702
Epoch 34		Train loss: 0.000437		Val loss 0.009690
Epoch 35		Train loss: 0.000495		Val loss 0.009821
Epoch 36		Train loss: 0.000429		Val loss 0.010008
Epoch 37		Train loss: 0.000561		Val loss 0.009904
Epoch 38		Train loss: 0.000493		Val loss 0.010194
Epoch 39		Train loss: 0.000419		Val loss 0.010187
Epoch 40		Train loss: 0.000365		Val loss 0.010304
Epoch 41		Train loss: 0.000330		Val loss 0.010157
Epoch 42		Train loss: 0.000384		Val loss 0.010516
Epoch 43		Train loss: 0.000382		Val loss 0.010336
Epoch 44		Train loss: 0.000399		Val loss 0.010295

Epoch 45 | Train loss: 0.000302 | Val loss 0.010517
Epoch 46 | Train loss: 0.000362 | Val loss 0.010401
Epoch 47 | Train loss: 0.000370 | Val loss 0.010337
Epoch 48 | Train loss: 0.000348 | Val loss 0.010483
Epoch 49 | Train loss: 0.000363 | Val loss 0.010611
Epoch 50 | Train loss: 0.000374 | Val loss 0.010574

Training model: hidden_size=32, num_heads=2, num_layers=4

Epoch 1 | Train loss: 2.737515 | Val loss 1.058072
Epoch 2 | Train loss: 0.578160 | Val loss 0.308781
Epoch 3 | Train loss: 0.182161 | Val loss 0.165740
Epoch 4 | Train loss: 0.100484 | Val loss 0.066990
Epoch 5 | Train loss: 0.063385 | Val loss 0.048923
Epoch 6 | Train loss: 0.050186 | Val loss 0.054901
Epoch 7 | Train loss: 0.038072 | Val loss 0.031228
Epoch 8 | Train loss: 0.033361 | Val loss 0.024756
Epoch 9 | Train loss: 0.024789 | Val loss 0.024396
Epoch 10 | Train loss: 0.024401 | Val loss 0.027963
Epoch 11 | Train loss: 0.019524 | Val loss 0.033637
Epoch 12 | Train loss: 0.015079 | Val loss 0.020762
Epoch 13 | Train loss: 0.014401 | Val loss 0.020684
Epoch 14 | Train loss: 0.011633 | Val loss 0.017755
Epoch 15 | Train loss: 0.009588 | Val loss 0.013853
Epoch 16 | Train loss: 0.008412 | Val loss 0.011452
Epoch 17 | Train loss: 0.006194 | Val loss 0.014119
Epoch 18 | Train loss: 0.010291 | Val loss 0.015895
Epoch 19 | Train loss: 0.007604 | Val loss 0.012385
Epoch 20 | Train loss: 0.005576 | Val loss 0.010797
Epoch 21 | Train loss: 0.004090 | Val loss 0.011433
Epoch 22 | Train loss: 0.003908 | Val loss 0.013410
Epoch 23 | Train loss: 0.003385 | Val loss 0.010087
Epoch 24 | Train loss: 0.002725 | Val loss 0.010080
Epoch 25 | Train loss: 0.003903 | Val loss 0.011662

Epoch 26 | Train loss: 0.004857 | Val loss 0.010549
Epoch 27 | Train loss: 0.003492 | Val loss 0.008404
Epoch 28 | Train loss: 0.001257 | Val loss 0.007251
Epoch 29 | Train loss: 0.001001 | Val loss 0.006317
Epoch 30 | Train loss: 0.000714 | Val loss 0.006194
Epoch 31 | Train loss: 0.000741 | Val loss 0.006263
Epoch 32 | Train loss: 0.000639 | Val loss 0.007138
Epoch 33 | Train loss: 0.000621 | Val loss 0.005978
Epoch 34 | Train loss: 0.000594 | Val loss 0.006595
Epoch 35 | Train loss: 0.000571 | Val loss 0.006695
Epoch 36 | Train loss: 0.000518 | Val loss 0.006570
Epoch 37 | Train loss: 0.000611 | Val loss 0.006364
Epoch 38 | Train loss: 0.001334 | Val loss 0.009323
Epoch 39 | Train loss: 0.004627 | Val loss 0.011998
Epoch 40 | Train loss: 0.002210 | Val loss 0.006199
Epoch 41 | Train loss: 0.000836 | Val loss 0.007613
Epoch 42 | Train loss: 0.000843 | Val loss 0.006731
Epoch 43 | Train loss: 0.000614 | Val loss 0.006491
Epoch 44 | Train loss: 0.000486 | Val loss 0.006397
Epoch 45 | Train loss: 0.000447 | Val loss 0.006228
Epoch 46 | Train loss: 0.000405 | Val loss 0.006354
Epoch 47 | Train loss: 0.000485 | Val loss 0.006111
Epoch 48 | Train loss: 0.000489 | Val loss 0.007304
Epoch 49 | Train loss: 0.000480 | Val loss 0.006377
Epoch 50 | Train loss: 0.000471 | Val loss 0.007082

Training model: hidden_size=24, num_heads=2, num_layers=4

Epoch 1 | Train loss: 1.972196 | Val loss 0.659611
Epoch 2 | Train loss: 0.337417 | Val loss 0.186947
Epoch 3 | Train loss: 0.155362 | Val loss 0.236331
Epoch 4 | Train loss: 0.107716 | Val loss 0.136227
Epoch 5 | Train loss: 0.078092 | Val loss 0.111972
Epoch 6 | Train loss: 0.062138 | Val loss 0.039964

Epoch 7 | Train loss: 0.052486 | Val loss 0.048194
Epoch 8 | Train loss: 0.042189 | Val loss 0.038937
Epoch 9 | Train loss: 0.034203 | Val loss 0.035236
Epoch 10 | Train loss: 0.032845 | Val loss 0.025965
Epoch 11 | Train loss: 0.030290 | Val loss 0.029647
Epoch 12 | Train loss: 0.023405 | Val loss 0.023020
Epoch 13 | Train loss: 0.025274 | Val loss 0.022195
Epoch 14 | Train loss: 0.020653 | Val loss 0.024236
Epoch 15 | Train loss: 0.020839 | Val loss 0.018550
Epoch 16 | Train loss: 0.017004 | Val loss 0.021831
Epoch 17 | Train loss: 0.015762 | Val loss 0.018706
Epoch 18 | Train loss: 0.013793 | Val loss 0.016511
Epoch 19 | Train loss: 0.012940 | Val loss 0.021757
Epoch 20 | Train loss: 0.012971 | Val loss 0.015275
Epoch 21 | Train loss: 0.010949 | Val loss 0.013699
Epoch 22 | Train loss: 0.010989 | Val loss 0.015414
Epoch 23 | Train loss: 0.008684 | Val loss 0.009995
Epoch 24 | Train loss: 0.010294 | Val loss 0.013518
Epoch 25 | Train loss: 0.008713 | Val loss 0.011777
Epoch 26 | Train loss: 0.007257 | Val loss 0.009522
Epoch 27 | Train loss: 0.006905 | Val loss 0.013918
Epoch 28 | Train loss: 0.007325 | Val loss 0.010078
Epoch 29 | Train loss: 0.006731 | Val loss 0.015526
Epoch 30 | Train loss: 0.005603 | Val loss 0.011458
Epoch 31 | Train loss: 0.004319 | Val loss 0.008548
Epoch 32 | Train loss: 0.003963 | Val loss 0.006670
Epoch 33 | Train loss: 0.003376 | Val loss 0.011382
Epoch 34 | Train loss: 0.004757 | Val loss 0.008772
Epoch 35 | Train loss: 0.003780 | Val loss 0.009231
Epoch 36 | Train loss: 0.003417 | Val loss 0.018316
Epoch 37 | Train loss: 0.002308 | Val loss 0.007462
Epoch 38 | Train loss: 0.003091 | Val loss 0.008999

Epoch 39 | Train loss: 0.002455 | Val loss 0.008109
Epoch 40 | Train loss: 0.002227 | Val loss 0.006804
Epoch 41 | Train loss: 0.003168 | Val loss 0.018327
Epoch 42 | Train loss: 0.002242 | Val loss 0.007397
Epoch 43 | Train loss: 0.001362 | Val loss 0.006405
Epoch 44 | Train loss: 0.001354 | Val loss 0.008680
Epoch 45 | Train loss: 0.002053 | Val loss 0.007321
Epoch 46 | Train loss: 0.001477 | Val loss 0.006805
Epoch 47 | Train loss: 0.001691 | Val loss 0.008361
Epoch 48 | Train loss: 0.001366 | Val loss 0.007355
Epoch 49 | Train loss: 0.000857 | Val loss 0.006783
Epoch 50 | Train loss: 0.000822 | Val loss 0.007603

Training model: hidden_size=16, num_heads=1, num_layers=4

Epoch 1 | Train loss: 2.009113 | Val loss 0.922187
Epoch 2 | Train loss: 0.574540 | Val loss 0.350170
Epoch 3 | Train loss: 0.296213 | Val loss 0.308396
Epoch 4 | Train loss: 0.188573 | Val loss 0.140835
Epoch 5 | Train loss: 0.142782 | Val loss 0.111947
Epoch 6 | Train loss: 0.109935 | Val loss 0.081158
Epoch 7 | Train loss: 0.090477 | Val loss 0.091084
Epoch 8 | Train loss: 0.081453 | Val loss 0.068548
Epoch 9 | Train loss: 0.070137 | Val loss 0.106395
Epoch 10 | Train loss: 0.066709 | Val loss 0.053084
Epoch 11 | Train loss: 0.058421 | Val loss 0.040655
Epoch 12 | Train loss: 0.056338 | Val loss 0.058005
Epoch 13 | Train loss: 0.048513 | Val loss 0.060449
Epoch 14 | Train loss: 0.044846 | Val loss 0.052096
Epoch 15 | Train loss: 0.048329 | Val loss 0.035347
Epoch 16 | Train loss: 0.040094 | Val loss 0.035727
Epoch 17 | Train loss: 0.036468 | Val loss 0.035124
Epoch 18 | Train loss: 0.035756 | Val loss 0.033388
Epoch 19 | Train loss: 0.041006 | Val loss 0.033129

Epoch 20		Train loss: 0.029055		Val loss 0.030997
Epoch 21		Train loss: 0.028350		Val loss 0.035500
Epoch 22		Train loss: 0.029781		Val loss 0.030075
Epoch 23		Train loss: 0.023883		Val loss 0.030999
Epoch 24		Train loss: 0.024717		Val loss 0.025368
Epoch 25		Train loss: 0.023600		Val loss 0.025514
Epoch 26		Train loss: 0.020803		Val loss 0.023735
Epoch 27		Train loss: 0.022144		Val loss 0.031747
Epoch 28		Train loss: 0.019800		Val loss 0.031171
Epoch 29		Train loss: 0.018045		Val loss 0.027775
Epoch 30		Train loss: 0.018419		Val loss 0.025204
Epoch 31		Train loss: 0.016574		Val loss 0.024749
Epoch 32		Train loss: 0.016973		Val loss 0.020959
Epoch 33		Train loss: 0.015602		Val loss 0.022006
Epoch 34		Train loss: 0.014246		Val loss 0.022224
Epoch 35		Train loss: 0.014461		Val loss 0.024670
Epoch 36		Train loss: 0.013671		Val loss 0.020617
Epoch 37		Train loss: 0.013179		Val loss 0.024854
Epoch 38		Train loss: 0.012587		Val loss 0.019916
Epoch 39		Train loss: 0.012191		Val loss 0.023161
Epoch 40		Train loss: 0.011675		Val loss 0.018328
Epoch 41		Train loss: 0.011658		Val loss 0.019577
Epoch 42		Train loss: 0.011450		Val loss 0.018223
Epoch 43		Train loss: 0.010649		Val loss 0.020797
Epoch 44		Train loss: 0.010019		Val loss 0.023445
Epoch 45		Train loss: 0.010495		Val loss 0.018124
Epoch 46		Train loss: 0.010106		Val loss 0.018552
Epoch 47		Train loss: 0.009661		Val loss 0.016004
Epoch 48		Train loss: 0.009116		Val loss 0.017598
Epoch 49		Train loss: 0.008931		Val loss 0.017807
Epoch 50		Train loss: 0.008984		Val loss 0.017875

Training model: hidden_size=16, num_heads=1, num_layers=3

Epoch 1 | Train loss: 2.271128 | Val loss 1.498162
Epoch 2 | Train loss: 1.327632 | Val loss 1.187578
Epoch 3 | Train loss: 1.055457 | Val loss 0.981754
Epoch 4 | Train loss: 0.891350 | Val loss 0.828685
Epoch 5 | Train loss: 0.722136 | Val loss 0.652050
Epoch 6 | Train loss: 0.546509 | Val loss 0.522823
Epoch 7 | Train loss: 0.404237 | Val loss 0.354816
Epoch 8 | Train loss: 0.317537 | Val loss 0.262290
Epoch 9 | Train loss: 0.246714 | Val loss 0.215585
Epoch 10 | Train loss: 0.206680 | Val loss 0.182680
Epoch 11 | Train loss: 0.169098 | Val loss 0.143272
Epoch 12 | Train loss: 0.147746 | Val loss 0.138300
Epoch 13 | Train loss: 0.128814 | Val loss 0.116952
Epoch 14 | Train loss: 0.112852 | Val loss 0.111209
Epoch 15 | Train loss: 0.099912 | Val loss 0.129305
Epoch 16 | Train loss: 0.092125 | Val loss 0.089660
Epoch 17 | Train loss: 0.080023 | Val loss 0.076146
Epoch 18 | Train loss: 0.076572 | Val loss 0.069141
Epoch 19 | Train loss: 0.069611 | Val loss 0.072696
Epoch 20 | Train loss: 0.064319 | Val loss 0.065495
Epoch 21 | Train loss: 0.058961 | Val loss 0.073029
Epoch 22 | Train loss: 0.054624 | Val loss 0.065433
Epoch 23 | Train loss: 0.052078 | Val loss 0.056910
Epoch 24 | Train loss: 0.046789 | Val loss 0.046027
Epoch 25 | Train loss: 0.044351 | Val loss 0.048916
Epoch 26 | Train loss: 0.043249 | Val loss 0.047232
Epoch 27 | Train loss: 0.042130 | Val loss 0.059235
Epoch 28 | Train loss: 0.039413 | Val loss 0.045365
Epoch 29 | Train loss: 0.038692 | Val loss 0.040141
Epoch 30 | Train loss: 0.037348 | Val loss 0.044432
Epoch 31 | Train loss: 0.034974 | Val loss 0.039952
Epoch 32 | Train loss: 0.031809 | Val loss 0.037018

Epoch 33 | Train loss: 0.031006 | Val loss 0.041421
Epoch 34 | Train loss: 0.030370 | Val loss 0.040365
Epoch 35 | Train loss: 0.030446 | Val loss 0.034898
Epoch 36 | Train loss: 0.028605 | Val loss 0.038838
Epoch 37 | Train loss: 0.028209 | Val loss 0.035368
Epoch 38 | Train loss: 0.027341 | Val loss 0.031115
Epoch 39 | Train loss: 0.026001 | Val loss 0.031514
Epoch 40 | Train loss: 0.024833 | Val loss 0.030776
Epoch 41 | Train loss: 0.024267 | Val loss 0.031414
Epoch 42 | Train loss: 0.023353 | Val loss 0.029667
Epoch 43 | Train loss: 0.022019 | Val loss 0.029298
Epoch 44 | Train loss: 0.021934 | Val loss 0.033857
Epoch 45 | Train loss: 0.021859 | Val loss 0.029238
Epoch 46 | Train loss: 0.021010 | Val loss 0.026258
Epoch 47 | Train loss: 0.019983 | Val loss 0.026984
Epoch 48 | Train loss: 0.019749 | Val loss 0.027037
Epoch 49 | Train loss: 0.019291 | Val loss 0.025815
Epoch 50 | Train loss: 0.018306 | Val loss 0.026277

Training model: hidden_size=12, num_heads=1, num_layers=3

Epoch 1 | Train loss: 2.558929 | Val loss 1.696074
Epoch 2 | Train loss: 1.593926 | Val loss 1.538081
Epoch 3 | Train loss: 1.475625 | Val loss 1.441326
Epoch 4 | Train loss: 1.397481 | Val loss 1.378920
Epoch 5 | Train loss: 1.336920 | Val loss 1.320578
Epoch 6 | Train loss: 1.261059 | Val loss 1.231259
Epoch 7 | Train loss: 1.150546 | Val loss 1.121369
Epoch 8 | Train loss: 1.021232 | Val loss 0.964343
Epoch 9 | Train loss: 0.899557 | Val loss 0.850432
Epoch 10 | Train loss: 0.814103 | Val loss 0.796974
Epoch 11 | Train loss: 0.740073 | Val loss 0.704051
Epoch 12 | Train loss: 0.662821 | Val loss 0.653908
Epoch 13 | Train loss: 0.616057 | Val loss 0.699221

Epoch 14		Train loss: 0.553758		Val loss 0.557838
Epoch 15		Train loss: 0.504468		Val loss 0.495371
Epoch 16		Train loss: 0.459063		Val loss 0.452649
Epoch 17		Train loss: 0.425693		Val loss 0.413719
Epoch 18		Train loss: 0.400792		Val loss 0.410217
Epoch 19		Train loss: 0.372065		Val loss 0.378410
Epoch 20		Train loss: 0.353387		Val loss 0.363992
Epoch 21		Train loss: 0.343761		Val loss 0.341342
Epoch 22		Train loss: 0.316849		Val loss 0.324487
Epoch 23		Train loss: 0.309238		Val loss 0.315153
Epoch 24		Train loss: 0.297731		Val loss 0.285062
Epoch 25		Train loss: 0.281099		Val loss 0.280972
Epoch 26		Train loss: 0.270660		Val loss 0.266432
Epoch 27		Train loss: 0.264326		Val loss 0.276598
Epoch 28		Train loss: 0.255754		Val loss 0.262845
Epoch 29		Train loss: 0.248254		Val loss 0.246470
Epoch 30		Train loss: 0.242684		Val loss 0.246571
Epoch 31		Train loss: 0.235276		Val loss 0.249953
Epoch 32		Train loss: 0.229437		Val loss 0.244684
Epoch 33		Train loss: 0.223398		Val loss 0.227077
Epoch 34		Train loss: 0.215237		Val loss 0.222133
Epoch 35		Train loss: 0.212989		Val loss 0.214277
Epoch 36		Train loss: 0.206605		Val loss 0.224266
Epoch 37		Train loss: 0.204257		Val loss 0.208041
Epoch 38		Train loss: 0.199669		Val loss 0.206769
Epoch 39		Train loss: 0.195192		Val loss 0.212819
Epoch 40		Train loss: 0.194341		Val loss 0.200990
Epoch 41		Train loss: 0.190412		Val loss 0.217512
Epoch 42		Train loss: 0.185940		Val loss 0.194374
Epoch 43		Train loss: 0.184209		Val loss 0.197882
Epoch 44		Train loss: 0.182005		Val loss 0.188820
Epoch 45		Train loss: 0.179322		Val loss 0.188440

Epoch 46 | Train loss: 0.175173 | Val loss 0.185568
Epoch 47 | Train loss: 0.173563 | Val loss 0.183965
Epoch 48 | Train loss: 0.170555 | Val loss 0.182668
Epoch 49 | Train loss: 0.170022 | Val loss 0.178881
Epoch 50 | Train loss: 0.169325 | Val loss 0.176719

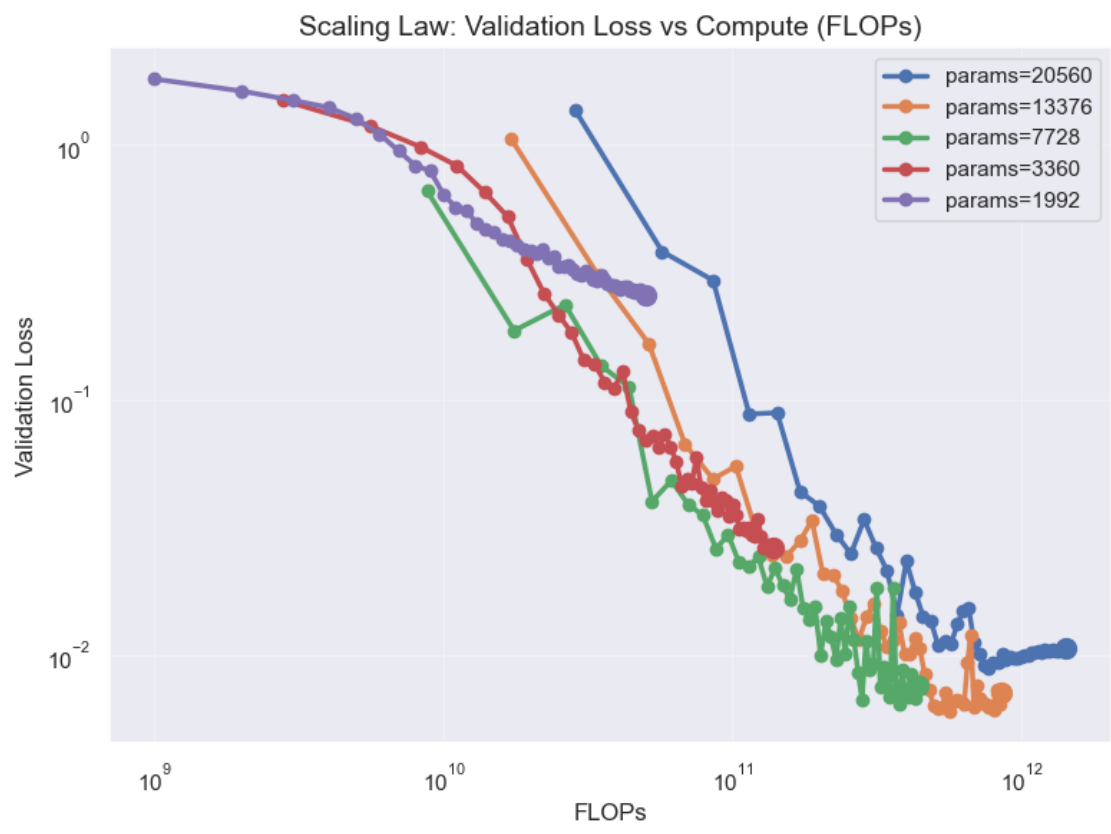
Training model: hidden_size=12, num_heads=1, num_layers=2

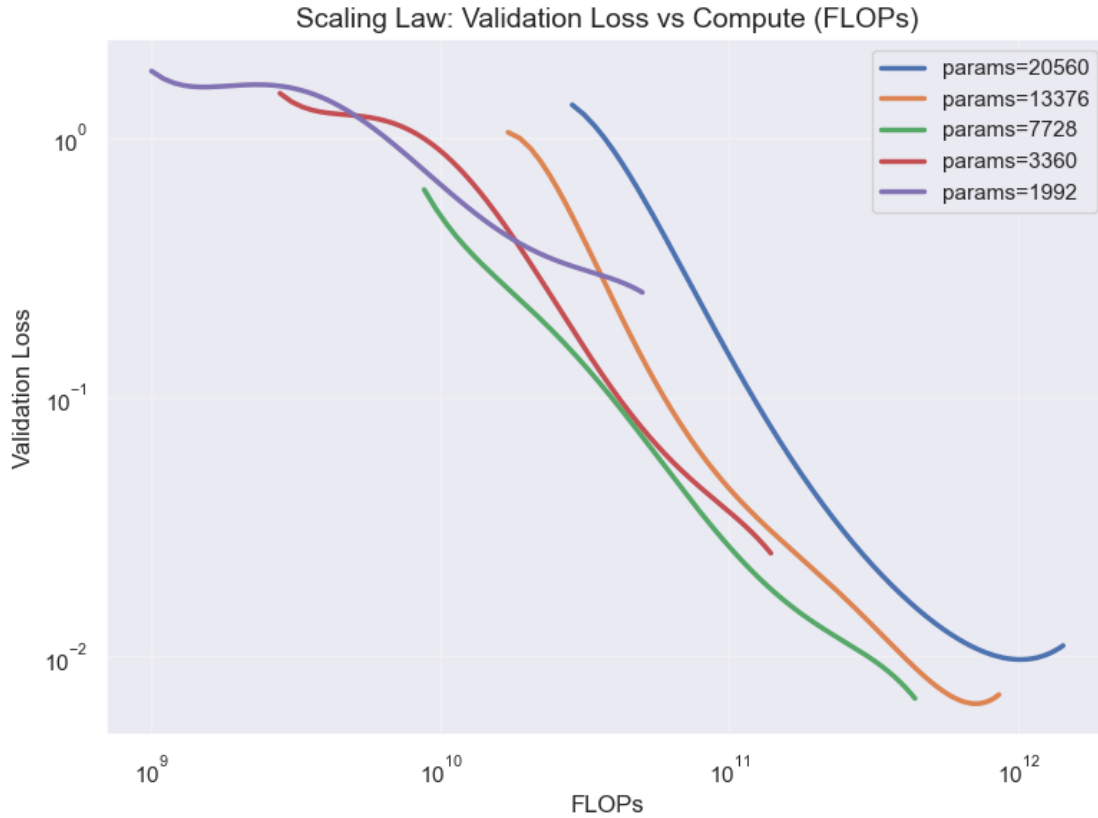
Epoch 1 | Train loss: 2.611423 | Val loss 1.814903
Epoch 2 | Train loss: 1.698923 | Val loss 1.628246
Epoch 3 | Train loss: 1.556196 | Val loss 1.498694
Epoch 4 | Train loss: 1.440351 | Val loss 1.403486
Epoch 5 | Train loss: 1.330254 | Val loss 1.268128
Epoch 6 | Train loss: 1.183388 | Val loss 1.105256
Epoch 7 | Train loss: 1.018010 | Val loss 0.947772
Epoch 8 | Train loss: 0.881584 | Val loss 0.829349
Epoch 9 | Train loss: 0.766185 | Val loss 0.792632
Epoch 10 | Train loss: 0.676621 | Val loss 0.640320
Epoch 11 | Train loss: 0.612511 | Val loss 0.570999
Epoch 12 | Train loss: 0.558083 | Val loss 0.553067
Epoch 13 | Train loss: 0.516073 | Val loss 0.494711
Epoch 14 | Train loss: 0.483439 | Val loss 0.467243
Epoch 15 | Train loss: 0.463845 | Val loss 0.453949
Epoch 16 | Train loss: 0.443285 | Val loss 0.426640
Epoch 17 | Train loss: 0.427971 | Val loss 0.422936
Epoch 18 | Train loss: 0.411110 | Val loss 0.405559
Epoch 19 | Train loss: 0.398902 | Val loss 0.389461
Epoch 20 | Train loss: 0.384422 | Val loss 0.385453
Epoch 21 | Train loss: 0.377262 | Val loss 0.372767
Epoch 22 | Train loss: 0.365701 | Val loss 0.388103
Epoch 23 | Train loss: 0.351205 | Val loss 0.360114
Epoch 24 | Train loss: 0.346069 | Val loss 0.365550
Epoch 25 | Train loss: 0.338993 | Val loss 0.333806
Epoch 26 | Train loss: 0.329828 | Val loss 0.333883

```
Epoch 27 | Train loss: 0.324681 | Val loss 0.338275
Epoch 28 | Train loss: 0.321309 | Val loss 0.325362
Epoch 29 | Train loss: 0.313190 | Val loss 0.314730
Epoch 30 | Train loss: 0.308772 | Val loss 0.308277
Epoch 31 | Train loss: 0.304952 | Val loss 0.320621
Epoch 32 | Train loss: 0.299663 | Val loss 0.309954
Epoch 33 | Train loss: 0.294892 | Val loss 0.297406
Epoch 34 | Train loss: 0.293509 | Val loss 0.291820
Epoch 35 | Train loss: 0.289838 | Val loss 0.310527
Epoch 36 | Train loss: 0.286311 | Val loss 0.295464
Epoch 37 | Train loss: 0.282835 | Val loss 0.286196
Epoch 38 | Train loss: 0.279698 | Val loss 0.281534
Epoch 39 | Train loss: 0.275256 | Val loss 0.281845
Epoch 40 | Train loss: 0.272824 | Val loss 0.278179
Epoch 41 | Train loss: 0.268394 | Val loss 0.271817
Epoch 42 | Train loss: 0.269716 | Val loss 0.279875
Epoch 43 | Train loss: 0.266807 | Val loss 0.277350
Epoch 44 | Train loss: 0.262992 | Val loss 0.269319
Epoch 45 | Train loss: 0.262022 | Val loss 0.271962
Epoch 46 | Train loss: 0.260807 | Val loss 0.265484
Epoch 47 | Train loss: 0.257847 | Val loss 0.265588
Epoch 48 | Train loss: 0.256842 | Val loss 0.269687
Epoch 49 | Train loss: 0.255257 | Val loss 0.261237
Epoch 50 | Train loss: 0.253043 | Val loss 0.257547
```

```
[39]: plot_scaling_law(all_results)

      plot_scaling_law_poly(all_results)
```





3.2 Step 2: Optimal Number of Parameters

Using the previously created training runs, we'll fit quadratic functions to the (parameter count, validation loss) pairs to estimate the optimal number of parameters for a given compute budget. We'll then fit a line to this set of optimal parameters to estimate the optimal number of parameters for a compute budget of $1e15$.

```
[40]: def find_optim_params(params: torch.Tensor, loss: torch.Tensor):
    x = np.array(params.cpu())
    y = np.array(loss.cpu())

    p = np.polyfit(np.log10(x), y, 2)
    optimal_log_params = -(p[1]) / (2 * p[0])
    optimal_params = 10 ** (optimal_log_params)

    return optimal_params

target_flops = [8e9, 16e9, 32e9, 64e9, 128e9]
colors = ["#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00"]
params_vs_loss = []
```



```

optimal_params = []

for target in target_flops:
    parameters = []
    val_loss = []

    for params, results in all_results.items():
        loss = interpolate(results["val_loss"], results["flops"], target, deg=4)

        # interpolate returns None if target is out of range
        if loss != None:
            parameters.append(params)
            val_loss.append(loss)

    sorted_indices = torch.argsort(torch.tensor(parameters)).to(device)
    parameters = torch.tensor(parameters).to(device)[sorted_indices]
    val_loss = torch.tensor(val_loss).to(device)[sorted_indices]
    try:
        optimal_params.append(find_optim_params(parameters, val_loss))
        params_vs_loss.append((parameters.tolist(), val_loss.tolist()))
    except Exception as e:
        print(
            "\n\n{optimal_params=}\n\n{params_vs_loss=}\n\n{parameters=}\n\n{val_loss=}\n\n{target=}"
        )

```

```

/var/folders/dj/k5kk5wk90vvcnb_wphpw490r0000gn/T/ipykernel_98922/2694499551.py:2
: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so
passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword
arguments. To learn more, see the migration guide
https://numpy.org/devdocs/numpy\_2\_0\_migration\_guide.html#adapting-to-changes-in-the-copy-keyword

```

```

x = np.array(params.cpu())
/var/folders/dj/k5kk5wk90vvcnb_wphpw490r0000gn/T/ipykernel_98922/2694499551.py:3
: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so
passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword
arguments. To learn more, see the migration guide
https://numpy.org/devdocs/numpy\_2\_0\_migration\_guide.html#adapting-to-changes-in-the-copy-keyword
y = np.array(loss.cpu())

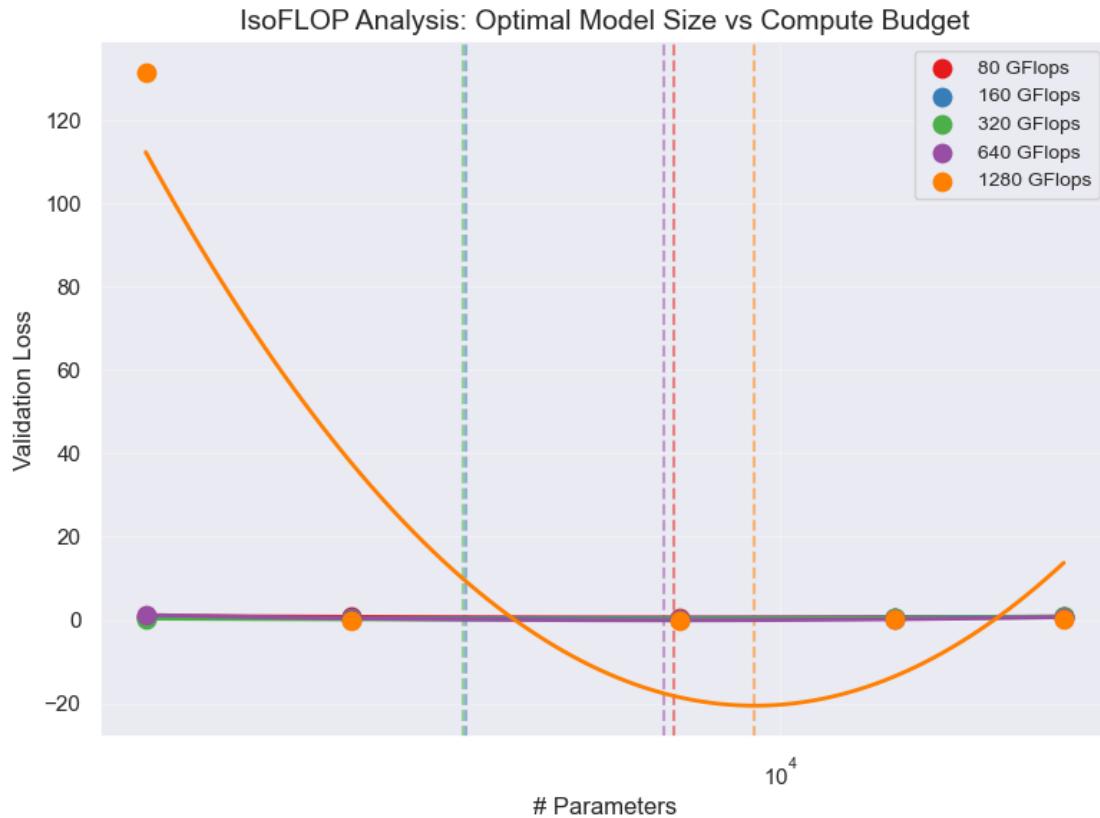
```

```
[41]: plot_isoflop(target_flops, colors, params_vs_loss, optimal_params)
```

```

8.00 GFlops: Optimal params = 7,633
16.00 GFlops: Optimal params = 4,499
32.00 GFlops: Optimal params = 4,472
64.00 GFlops: Optimal params = 7,435
128.00 GFlops: Optimal params = 9,365

```



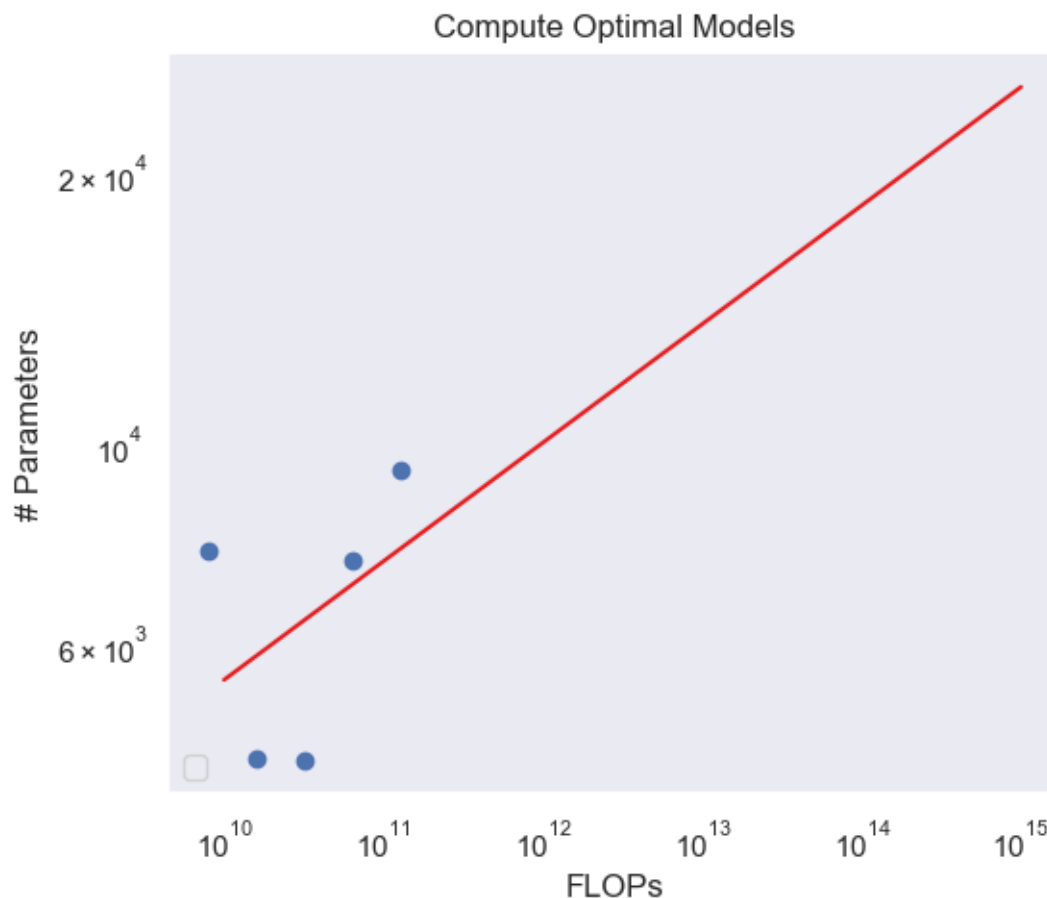
```
[ ]: plot_flops_params(target_flops, optimal_params)
```

```
y = 0.1314992772087517x + 2.424400231521431
```

```
/Users/Akseldkw/Desktop/Columbia/COMS4776/homework/HW2/utils.py:282:
```

```
UserWarning: No artists with labels found to put in legend. Note that artists
whose label start with an underscore are ignored when legend() is called with no
argument.
```

```
ax.legend(loc="lower left")
```



3.3 Step 3: Optimal Number of Tokens

Finally, we'll perform a similar analysis to the previous step, only this time determining the optimal number of tokens for a budget of $1e15$ flops.

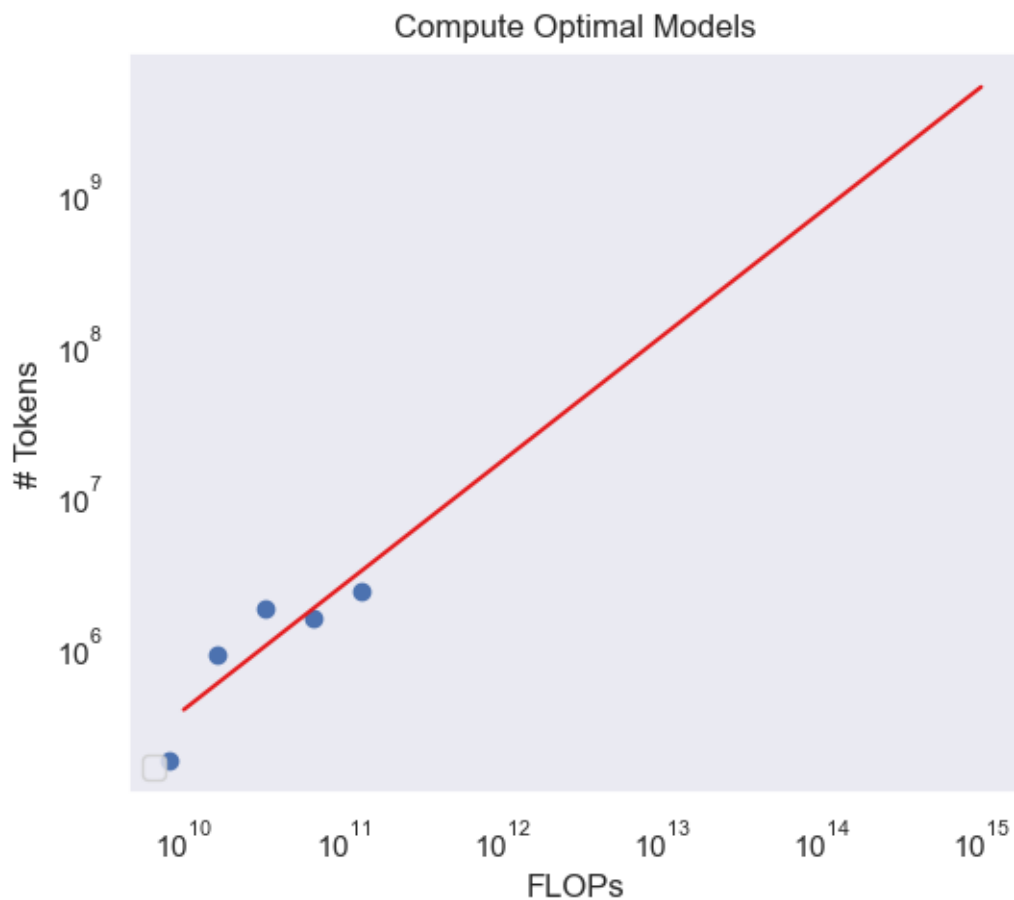
```
[43]: plot_flops_tokens(target_flops, optimal_params, all_results)
```

```
y = 0.824755800807312x + -2.671545790358438
```

```
/Users/Akseldkw/Desktop/Columbia/COMS4776/homework/HW2/utils.py:323:
```

```
UserWarning: No artists with labels found to put in legend. Note that artists
whose label start with an underscore are ignored when legend() is called with no
argument.
```

```
ax.legend(loc="lower left")
```



3.3.1 Step 4: Analyzing results

1. Flops vs. # Parameters: It looks like the relationship is quadratic. The graph is hard to interpret because the scale is dominated by the large GFlop values, but it seems like there's a local minimum for the ideal number of parameters for a given compute budget.
2. Flops vs. # Tokens: Unlike the previous graph, this one looks more linear. It seems like the more tokens we have, the better, and there isn't a clear local minima.